DEVOPS ENGINEERING MASTERY

GIT, SCRIPTING & AUTOMATION ENGINEERING

Git, Bash, Python & Software Automation

VOLUME 03 • FIRST EDITION

```
nahid@ubuntu-server:~$ git log --oneline --graph
* a1b2c3d (HEAD -> main) Add health-check script
* 9f8e7d6 Merge feature/backup-automation
nahid@ubuntu-server:~$ ./health-check.sh
OK: disk 42% • memory 61% • nginx running
nahid@ubuntu-server:~$ python3 -m pytest tests/ -q
```

Ubuntu Server 24.04 LTS (Noble Numbat) — Primary hands-on environment

Git 2.43 • Bash 5.2 • Python 3.12 • pytest 9.2

First Edition • Published 8 November 2026 • Dhaka, Bangladesh

Companion Volume • 120 MCQs with Explanations • Final Exam

Quiz & Answer Guide

DEVOPS ENGINEERING MASTERY — Volume 03, Companion Volume. First Edition, 8 November 2026. This guide gives the answers to all 120 chapter-end MCQs *with explanations* — why the correct answer is correct and why each plausible wrong option is wrong — plus model answers for short-answer questions, Git command-interpretation and Bash code-reading exercises, Python debugging exercises, script-writing challenges, troubleshooting case studies, the practical final examination with marking rubric, a 197-term glossary and 30 official references. Attempt every question yourself before reading its answer.

> **WARNING:** Do not copy assignment solutions word for word. Read the outline, close this guide, and do the work yourself in your VM. That is where the learning happens.

পরিশিষ্ট D • Quiz and Final Examination Answer Guide

Appendix D: Quiz and Final Examination Answer Guide — কুইজ ও চূড়ান্ত পরীক্ষার উত্তর-নির্দেশিকা

এই পরিশিষ্টে বইয়ের ২৪টি অধ্যায়ের সব কুইজের উত্তর দেওয়া হয়েছে — মোট ১২০টি MCQ। প্রতিটি প্রশ্নের সঙ্গে সঠিক উত্তরের ব্যাখ্যা এবং প্রতিটি ভুল বিকল্প কেন ভুল, তা বাংলায় বলা হয়েছে। শুধু উত্তর মুখস্থ করবেন না; ব্যাখ্যাটি পড়ে ধারণাটি বুঝুন। ভুল বিকল্পগুলো ইচ্ছাকৃতভাবে এমনভাবে লেখা হয়েছে, যাতে সাধারণ ভুল ধারণাগুলো ধরা পড়ে। পরীক্ষার পরিবেশ: Ubuntu Server 24.04 LTS, Git 2.43, Bash 5.2, Python 3.12। এই সংস্করণগুলোতে যাচাই করা আচরণই এখানে সঠিক বলে ধরা হয়েছে।

> **CONCEPT:** ভালো MCQ-র ভুল বিকল্পও শিক্ষণীয়। প্রতিটি ব্যাখ্যায় তিনটি প্রশ্নের উত্তর খুঁজুন — সঠিকটি কেন সঠিক, ভুলটি কেন ভুল, এবং বাস্তব কাজে পার্থক্যটি কী প্রভাব ফেলে।

D.1 অধ্যায় ১ — Introduction to Version Control Systems (প্রশ্ন ১.১–১.৫)

১.১ Version control-এর প্রধান উদ্দেশ্য কোনটি?
ক) ফাইল দ্রুত কপি করা খ) সময়ের সঙ্গে সঙ্গে পরিবর্তনের ইতিহাস সংরক্ষণ করা ও প্রয়োজনে আগের অবস্থায় ফেরা গ) ফাইল compress করা ঘ) ইন্টারনেট ছাড়া ফাইল শেয়ার করা
উত্তর: খ — version control প্রতিটি পরিবর্তন কে, কখন, কেন করল তা ইতিহাসসহ সংরক্ষণ করে, ফলে যেকোনো আগের অবস্থায় ফেরা যায়। ক ভুল, কারণ কপি করা তার কাজ নয়; গ ভুল, কারণ compression এর উদ্দেশ্য নয়; ঘ ভুল, কারণ শেয়ার করা গৌণ সুবিধা, মূল উদ্দেশ্য ইতিহাস সংরক্ষণ।

১.২ Manual backup (যেমন project-v1, project-v2 ফোল্ডার)-এর সবচেয়ে বড় দুর্বলতা কী?
ক) বেশি জায়গা নেয় খ) কে কী পরিবর্তন করল, কেন করল — তার কোনো নির্ভরযোগ্য রেকর্ড থাকে না গ) ফোল্ডারের নাম লম্বা হয় ঘ) এটি শুধু Linux-এ চলে
উত্তর: খ — manual কপিতে পরিবর্তনের কারণ, লেখক ও পার্থক্যের কোনো কাঠামোবদ্ধ রেকর্ড থাকে না, ফলে দুটি কপি

মেলানো কঠিন হয়। ক আংশিক সত্য হলেও প্রধান সমস্যা নয়; গ তুচ্ছ; ঘ ভুল, কারণ manual backup সব OS-এই একইভাবে দুর্বল।

১.৩ Git কোন ধরনের version control system?
ক) Local-only খ) Centralized গ) Distributed ঘ) Cloud-only
উত্তর: গ — Git distributed: প্রতিটি clone-এ পূর্ণ ইতিহাসসহ repository থাকে, সার্ভার ছাড়াও commit করা যায়। ক ভুল, কারণ Git remote-এর সঙ্গে কাজ করে; খ ভুল, কারণ centralized ব্যবস্থায় (যেমন পুরনো SVN) ইতিহাস শুধু সার্ভারে থাকে; ঘ ভুল, কারণ Git offline-এও চলে।

১.৪ Git ও GitHub-এর পার্থক্য কোনটি সঠিক?
ক) দুটি একই জিনিস খ) Git একটি command-line tool ও version control ব্যবস্থা; GitHub হলো Git repository হোস্ট করার একটি সেবা গ) GitHub ছাড়া Git চলে না ঘ) Git শুধু ওয়েবসাইট, GitHub শুধু কমান্ড
উত্তর: খ — Git আপনার কম্পিউটারে চলে; GitHub সেই repository ইন্টারনেটে রাখে ও collaboration সুবিধা (pull request, issue) দেয়। ক ভুল, কারণ দুটি আলাদা স্তর; গ ভুল, কারণ Git সম্পূর্ণ offline-এ কাজ করে; ঘ উল্টো বলা হয়েছে।

১.৫ একই ফাইলে দুজন developer আলাদা অংশ বদলালে version control কী করে?
ক) একজনের কাজ মুছে দেয় খ) দুজনের পরিবর্তন একত্র করে (merge), আর একই লাইনে সংঘাত হলে conflict দেখিয়ে মানুষকে সিদ্ধান্ত নিতে দেয় গ) ফাইলটি lock করে দেয় ঘ) দুটি আলাদা ফাইল বানিয়ে দেয়
উত্তর: খ — Git আলাদা লাইনের পরিবর্তন স্বয়ংক্রিয়ভাবে মেলায়; একই লাইনে দুই রকম পরিবর্তন হলে conflict চিহ্নিত করে, কারণ কোনটি রাখা হবে তা যন্ত্র সিদ্ধান্ত নিতে পারে না। ক ভুল, কারণ Git তথ্য হারায় না; গ ভুল, কারণ Git lock-ভিত্তিক নয়; ঘ ভুল, কারণ ফাইল ভাগ হয় না।

D.2 অধ্যায় ২ — Git Architecture and Internal Working (প্রশ্ন ২.১–২.৫)

২.১ Git-এ একটি blob object কী সংরক্ষণ করে?
ক) ফাইলের নাম ও directory কাঠামো খ) একটি ফাইলের বিষয়বস্তু (content), নাম ছাড়া গ) commit বার্তা ঘ) branch-এর নাম
উত্তর: খ — blob শুধু ফাইলের বিষয়বস্তু ধারণ করে; নাম ও কাঠামো থাকে tree object-এ। ক ভুল, সেটি tree-র কাজ; গ ভুল, সেটি commit object-এ থাকে; ঘ ভুল, branch হলো reference, object নয়।

২.২ Tree object-এর কাজ কী?
ক) ফাইলের বিষয়বস্তু রাখা খ) একটি directory-র তালিকা — কোন নামে কোন blob বা sub-tree আছে, তা ধারণ করা গ) লেখকের নাম রাখা ঘ) remote-এর ঠিকানা রাখা
উত্তর: খ — tree হলো directory snapshot: প্রতিটি entry-তে নাম, mode ও সংশ্লিষ্ট blob/tree-র hash থাকে। ক blob-র কাজ; গ commit-এর metadata; ঘ remote config-এ থাকে, object-এ নয়।

২.৩ Commit object-এ কী কী থাকে?
ক) শুধু বার্তা খ) tree-র hash, parent commit-এর hash, লেখক ও committer, সময় ও বার্তা গ) শুধু ফাইলের নাম ঘ) branch ও remote-এর তালিকা
উত্তর: খ — commit একটি snapshot-কে (tree) তার পূর্ববর্তী commit-এর সঙ্গে যুক্ত করে এবং কে কখন লিখল তা রেকর্ড করে। ক অসম্পূর্ণ; গ tree-র কাজ; ঘ config/reference-এর বিষয়।

২.৪ HEAD আসলে কী?
ক) প্রথম commit খ) বর্তমান branch-কে নির্দেশ করা একটি pointer — সাধারণত বর্তমান branch-এর দিকে, যার মাধ্যমে বর্তমান commit নির্ধারিত হয় গ) remote repository ঘ) staging area-র আরেক নাম

উত্তর: খ — HEAD সাধারণত `refs/heads/main`-এর দিকে নির্দেশ করে; detached অবস্থায় সরাসরি কোনো commit-কে নির্দেশ করে। ক ভুল, HEAD প্রথম commit নয়; গ ভুল, remote আলাদা; ঘ ভুল, staging area হলো index।

২.৫ Staging area (index)-এর ভূমিকা কী?
ক) ফাইল মুছে ফেলা খ) পরের commit-এ ঠিক কোন পরিবর্তনগুলো যাবে, তা বেছে নেওয়ার জায়গা গ) remote-এ পাঠানো ঘ) পুরনো commit দেখা
উত্তর: খ — `git add` পরিবর্তন staging area-তে তোলে; commit শুধু সেখানকার বিষয়বস্তু snapshot করে। ক ভুল; গ push-এর কাজ; ঘ log/show-এর কাজ।

D.3 অধ্যায় ৩ — Git Repositories, Commits and History (প্রশ্ন ৩.১–৩.৫)

৩.১ `git status` কী দেখায়?
ক) সব commit-এর তালিকা খ) working directory ও staging area-র বর্তমান অবস্থা — কোন ফাইল বদলেছে, staged হয়েছে বা untracked আছে গ) remote-এর ঠিকানা ঘ) ফাইলের ভেতরের পার্থক্য লাইন ধরে
উত্তর: খ — status হলো কাজের অবস্থার সারসংক্ষেপ। ক `git log`-এর কাজ; গ `git remote -v`-এর কাজ; ঘ `git diff`-এর কাজ।

৩.২ `git add file.txt` কী করে?
ক) ফাইলটি remote-এ পাঠায় খ) ফাইলটির বর্তমান বিষয়বস্তু staging area-তে তোলে গ) ফাইলটি commit করে ঘ) ফাইলটি মুছে দেয়
উত্তর: খ — add শুধু stage করে; commit আলাদা ধাপ। ক push-এর কাজ; গ commit-এর কাজ; ঘ `git rm`-এর কাজ।

৩.৩ `git log --oneline` কী দেখায়?
ক) ফাইলের পার্থক্য খ) প্রতিটি commit এক লাইনে — সংক্ষিপ্ত hash ও বার্তা গ) branch-এর তালিকা ঘ) untracked ফাইল
উত্তর: খ — `--oneline` ইতিহাস সংক্ষেপে দেখায়। ক diff-এর কাজ; গ `git branch`-এর কাজ; ঘ status-এ দেখা যায়।

৩.৪ `.gitignore` ফাইলের কাজ কী?
ক) Git বন্ধ করা খ) যেসব ফাইল/প্যাটার্ন Git track করবে না, তা নির্ধারণ করা (যেমন log, build output, `.env`) গ) ফাইল মুছে ফেলা ঘ) commit বার্তা লেখা
উত্তর: খ — .gitignore-এ লেখা প্যাটার্ন untracked থাকে। ক ভুল; গ ভুল, ফাইল disk থেকে মোছে না; ঘ ভুল। মনে রাখবেন: already tracked ফাইলকে .gitignore স্বয়ংক্রিয়ভাবে untrack করে না — আগে `git rm --cached` লাগে।

৩.৫ ভালো commit বার্তার বৈশিষ্ট্য কোনটি?
ক) "fix", "update" লেখা খ) সংক্ষিপ্ত imperative শিরোনামে কী পরিবর্তন ও কেন, তা বোঝানো, যেমন "Add health check to backup script" গ) যত লম্বা হয় তত ভালো ঘ) বার্তায় password লেখা
উত্তর: খ — ভালো বার্তা ভবিষ্যতের পাঠককে পরিবর্তনের কারণ বোঝায়। ক অস্পষ্ট; গ লম্বা হওয়া গুণ নয়; ঘ নিরাপত্তা লঙ্ঘন — secret কখনো commit করা যায় না।

D.4 অধ্যায় ৪ — Branching, Merging and Rebasing (প্রশ্ন ৪.১–৪.৫)

৪.১ Branch আসলে কী?
ক) ফাইলের কপি খ) একটি commit-কে নির্দেশ করা হালকা pointer (reference) গ) আলাদা repository ঘ) ফোল্ডার
উত্তর: খ — branch তৈরি করা সস্তা, কারণ এটি শুধু একটি hash সংরক্ষণ করে; ফাইল কপি হয় না। ক ভুল, কপি নয়; গ ভুল; ঘ ভুল।

৪.২ Fast-forward merge কখন হয়?
ক) conflict থাকলে খ) main branch-এ feature branch তৈরি হওয়ার পর main-এ কোনো নতুন commit না থাকলে, তখন main-এর pointer-কে শুধু এগিয়ে দেওয়া হয় গ) সবসময় ঘ) rebase করলে
উত্তর: খ — fast-forward-এ কোনো merge commit তৈরি হয় না, pointer এগিয়ে যায়। ক ভুল, conflict হলে fast-forward হয় না; গ ভুল; ঘ ভুল, rebase আলাদা কৌশল।

৪.৩ Merge conflict চিহ্ন `<<<<<<<` কী বোঝায়?
ক) ফাইল নষ্ট হয়েছে খ) একই লাইনে দুই branch-এর ভিন্ন পরিবর্তন আছে — মানুষকে বেছে নিয়ে চিহ্ন মুছে সমাধান করতে হবে গ) ফাইলটি মুছে গেছে ঘ) Git বন্ধ হয়ে গেছে
উত্তর: খ — conflict marker দুটি সংস্করণ দেখায়; সমাধানের পর `git add` করে merge শেষ করতে হয়। ক ভুল, ফাইল নষ্ট হয়নি; গ ভুল; ঘ ভুল।

৪.৪ Rebase ও merge-এর মূল পার্থক্য কী?
ক) কোনো পার্থক্য নেই খ) merge ইতিহাস যেমন আছে তেমন রেখে merge commit যোগ করে; rebase commit-গুলোকে নতুন base-এর উপর আবার প্রয়োগ করে সরলরৈখিক ইতিহাস বানায় গ) rebase conflict তৈরি করে না ঘ) merge branch মুছে দেয়
উত্তর: খ — rebase ইতিহাস পুনর্লিখন করে, তাই public/shared branch-এ rebase বিপজ্জনক। ক ভুল; গ ভুল, rebase-এও conflict হয়; ঘ ভুল, merge branch মোছে না।

৪.৫ `git reset --hard HEAD~1` কী করে?
ক) শুধু commit বার্তা বদলায় খ) HEAD, index ও working directory — তিনটিই আগের commit-এর অবস্থায় ফিরিয়ে দেয় এবং পরের commit-এর পরিবর্তন হারিয়ে যায় (uncommitted ও ওই commit-এর কাজ) গ) নতুন branch বানায় ঘ) remote-এ পাঠায়
উত্তর: খ — `--hard` সব স্তর বদলায়, তাই shared ইতিহাসে বা না বুঝে ব্যবহার বিপজ্জনক; নিরাপদ বিকল্প `git revert`। ক `git commit --amend`-এর কাজ; গ ভুল; ঘ ভুল।

D.5 অধ্যায় ৫ — Remote Repositories and GitHub (প্রশ্ন ৫.১–৫.৫)

৫.১ `git fetch` ও `git pull`-এর পার্থক্য কী?
ক) একই কাজ খ) fetch শুধু remote-এর পরিবর্তন ডাউনলোড করে, working directory বদলায় না; pull = fetch + merge (বা rebase) গ) fetch push করে ঘ) pull শুধু ডাউনলোড করে
উত্তর: খ — fetch নিরাপদে দেখার সুযোগ দেয়, pull সরাসরি মেলায়। ক ভুল; গ ভুল; ঘ উল্টো বলা হয়েছে।

৫.২ Tracking branch কী?
ক) মুছে যাওয়া branch খ) local branch, যার সঙ্গে একটি নির্দিষ্ট remote branch যুক্ত থাকে, ফলে `git pull` / `git push` জানে কোথা থেকে আনতে/কোথায় পাঠাতে হবে গ) main branch ঘ) tag
উত্তর: খ — `git push -u origin feature` upstream সেট করে। ক ভুল; গ ভুল, main-ও একটি tracking branch হতে পারে, কিন্তু সংজ্ঞা সেটি নয়; ঘ ভুল, tag স্থির pointer।

৫.৩ GitHub-এ SSH authentication-এর জন্য কী লাগে?
ক) password ইমেইল করা খ) নিজের কম্পিউটারে key pair তৈরি করে public key GitHub-এ যোগ করা; private key কখনো শেয়ার করা যাবে না গ) token ফাইলে লেখা ঘ) কিছুই লাগে না
উত্তর: খ — SSH-তে private key দিয়ে পরিচয় প্রমাণিত হয়, password পাঠানো হয় না। ক অনিরাপদ ও ভুল; গ token HTTPS-এর পদ্ধতি, SSH-এর নয়; ঘ ভুল।

৫.৪ Pull request (PR) আসলে কী?
ক) ফাইল ডাউনলোডের অনুরোধ খ) একটি branch-এর পরিবর্তন অন্য branch-এ merge করার প্রস্তাব, যেখানে review, আলোচনা ও পরীক্ষা হয় গ) repository মুছে ফেলার অনুরোধ ঘ) issue বন্ধ করা
উত্তর: খ — PR merge করার আগে code review-এর জায়গা। ক ভুল; গ ভুল; ঘ ভুল, PR merge হলে সংশ্লিষ্ট issue বন্ধ হতে পারে, কিন্তু সেটি সংজ্ঞা নয়।

৫.৫ Tag ও release-এর সম্পর্ক কী?
ক) একই জিনিস খ) tag একটি নির্দিষ্ট commit-কে চিহ্নিত করে (যেমন v1.0.0); release হলো সেই tag-এর উপর ভিত্তি করে GitHub-এ প্রকাশিত প্যাকেজ/নোটসহ সংস্করণ গ) tag branch-এর নাম ঘ) release commit মুছে দেয়
উত্তর: খ — tag Git-এর স্থির pointer, release GitHub-এর উপস্থাপনা স্তর। ক ভুল; গ ভুল; ঘ ভুল।

D.6 অধ্যায় ৬ — Professional Git Workflow and Collaboration (প্রশ্ন ৬.১–৬.৫)

৬.১ Feature branch workflow-এ প্রতিটি নতুন কাজ কোথায় করা হয়?
ক) সরাসরি main-এ খ) main থেকে আলাদা একটি ছোট branch-এ, শেষে PR-এর মাধ্যমে main-এ merge করা হয় গ) আলাদা repository-তে ঘ) local ফাইলে, commit ছাড়া
উত্তর: খ — এতে main সবসময় স্থিতিশীল থাকে এবং প্রতিটি পরিবর্তন review হয়। ক trunk-based-এর কাছাকাছি, কিন্তু feature workflow-এর সংজ্ঞা নয়; গ অপ্রয়োজনীয়; ঘ version control-এর উদ্দেশ্যের বিপরীত।

৬.২ GitHub Flow-এর মূল ধাপগুলো কী?
ক) main-এ সরাসরি push খ) branch তৈরি → commit → pull request → review → merge → deploy গ) fork ছাড়া কিছু নয় ঘ) tag তৈরি করা
উত্তর: খ — GitHub Flow-এ main সবসময় deployable থাকে। ক ভুল, কারণ review বাদ পড়ে; গ ভুল; ঘ ভুল, tag একটি ধাপ মাত্র।

৬.৩ Semantic versioning 2.3.1-এ সংখ্যাগুলো কী বোঝায়?
ক) বছর.মাস.দিন খ) major.minor.patch — major = ভাঙা পরিবর্তন, minor = পেছনে-সামঞ্জস্যপূর্ণ নতুন ফিচার, patch = bug fix গ) লেখকের সংখ্যা ঘ) ফাইলের সংখ্যা
উত্তর: খ — semver-এর এটাই সংজ্ঞা। ক ভুল, সেটি date-based versioning; গ ও ঘ ভুল।

৬.৪ Protected branch কেন ব্যবহার করা হয়?
ক) branch লুকাতে খ) main-এ সরাসরি push বন্ধ করে PR review ও পরীক্ষা বাধ্যতামূলক করতে গ) branch দ্রুত করতে ঘ) ফাইল compress করতে
উত্তর: খ — protected branch দুর্ঘটনাবশত বা review ছাড়া পরিবর্তন ঠেকায়। ক ভুল; গ ভুল; ঘ ভুল।

৬.৫ ভালো pull request-এ কী থাকা উচিত?
ক) শুধু কোড খ) কী বদলেছে, কেন বদলেছে, কীভাবে পরীক্ষা করা হয়েছে — তার সংক্ষিপ্ত বিবরণ ও ছোট, পর্যালোচনাযোগ্য পরিবর্তন গ) অনেকগুলো অসম্পর্কিত পরিবর্তন একসঙ্গে ঘ) কোনো বিবরণ ছাড়া বড় diff
উত্তর: খ — ছোট ও বর্ণনাসহ PR দ্রুত review হয়। ক অসম্পূর্ণ; গ review কঠিন করে; ঘ reviewer-কে অনুমান করতে বাধ্য করে।

D.7 অধ্যায় ৭ — Introduction to Shell Scripting (প্রশ্ন ৭.১–৭.৫)

৭.১ Shebang (`#!/bin/bash`) কী?
ক) মন্তব্য খ) script-এর প্রথম লাইনে লেখা নির্দেশ, যা বলে কোন interpreter দিয়ে script চালাতে হবে গ) ফাইলের নাম

ঘ) ভেরিয়েবল

উত্তর: খ — kernel shebang পড়ে `/bin/bash` দিয়ে script চালায়। ক ভুল, `#` দিয়ে শুরু হলেও shebang বিশেষ অর্থ বহন করে; গ ভুল; ঘ ভুল।

৭.২ একটি script-কে execute করার জন্য কী লাগে?

ক) শুধু লেখা খ) execute permission (`chmod +x`) এবং সঠিক shebang, অথবা `bash script.sh` দিয়ে চালানো গ) root হওয়া ঘ) ইন্টারনেট

উত্তর: খ — সরাসরি চালাতে `+x` লাগে; `bash script.sh` দিলে permission ছাড়াও চলে, কারণ তখন bash-ই interpreter। ক অসম্পূর্ণ; গ ভুল, root সব script-এর জন্য লাগে না; ঘ ভুল।

৭.৩ Shell ও terminal-এর পার্থক্য কী?

ক) একই জিনিস খ) terminal হলো window/প্রোগ্রাম যেখানে লেখা দেখা যায়; shell (Bash) হলো সেই লেখা পড়ে কমান্ড চালানো প্রোগ্রাম গ) shell শুধু GUI ঘ) terminal শুধু ফাইল

উত্তর: খ — terminal input/output-এর মাধ্যম, shell নির্দেশনা ব্যাখ্যা করে। ক ভুল; গ ও ঘ ভুল।

৭.৪ `bash -x script.sh` কী করে?

ক) script মুছে দেয় খ) প্রতিটি কমান্ড চালানোর আগে তা দেখিয়ে debug করতে সাহায্য করে (trace mode) গ) script দ্রুত চালায় ঘ) ভেরিয়েবল মুছে দেয়

উত্তর: খ — `-x` প্রতিটি কমান্ড `+` চিহ্নসহ দেখায়। ক ভুল; গ ভুল; ঘ ভুল।

৭.৫ Interactive ও non-interactive shell-এর পার্থক্য কী?

ক) কোনো পার্থক্য নেই খ) interactive shell মানুষের সঙ্গে সরাসরি কথা বলে (prompt দেখায়, input নেয়); script চালানো shell non-interactive গ) non-interactive দ্রুত নয় ঘ) interactive-এ script চলে না

উত্তর: খ — interactive shell prompt ও history দেয়; script-এর shell input ফাইল থেকে পড়ে। ক ভুল; গ ভুল; ঘ ভুল, interactive shell-এও script চালানো যায়।

D.8 অধ্যায় ৮ — Variables, Data Types and User Input (প্রশ্ন ৮.১–৮.৫)

৮.১ Bash-এ `name="Nahid"` লেখার সময় `=`-এর আশেপাশে space দিলে কী হয়?

ক) কিছু হয় না খ) Bash `name`-কে কমান্ড ভেবে ভুল করে (command not found) অথবা ভুল ব্যাখ্যা করে — assignment-এ space দেওয়া যায় না গ) ভেরিয়েবল মুছে যায় ঘ) script দ্রুত চলে

উত্তর: খ — Bash-এ assignment-এ `=`-এর দুপাশে space থাকলে সেটি আর assignment থাকে না। ক ভুল; গ ভুল; ঘ ভুল।

৮.২ `"$name"` ও `$name`-এর মধ্যে নিরাপদ কোনটি এবং কেন?

ক) দুটো একই খ) double quote করা রূপ নিরাপদ, কারণ space বা glob থাকা মান ভেঙে একাধিক শব্দ হয় না গ) quote ছাড়া নিরাপদ ঘ) single quote-এ ভেরিয়েবল expand হয়

উত্তর: খ — unquoted expansion word splitting ও globbing ঘটায়, যা ফাঁকা বা `*` থাকা মানে ভুল আচরণ করে। ক ভুল; গ ভুল; ঘ ভুল — single quote-এ `$name` আক্ষরিক থাকে, expand হয় না।

৮.৩ `$?`, `$#`, `$@` — এগুলো কী?

ক) সবই একই খ) `$?` শেষ কমান্ডের exit status, `$#` argument-এর সংখ্যা, `$@` সব argument গ) `$?` হলো process ID ঘ) `$#` হলো script-এর নাম

উত্তর: খ — এটাই সঠিক সংজ্ঞা। গ ভুল, process ID হলো `$$`; ঘ ভুল, script-এর নাম `$0`।

৮.৪ `export API_URL="https://example.com"` কী করে?
ক) ফাইলে লেখে খ) ভেরিয়েবলটিকে child process-গুলোর কাছে পৌঁছে দেয় (environment variable বানায়) গ) ভেরিয়েবল মুছে দেয় ঘ) ইন্টারনেটে পাঠায়
উত্তর: খ — export ছাড়া ভেরিয়েবল শুধু বর্তমান shell-এ থাকে; export করলে child process-ও পায়। ক ভুল; গ ভুল; ঘ ভুল।

৮.৫ Command substitution `$(date)` কী করে?
ক) কমান্ড মুছে দেয় খ) কমান্ড চালিয়ে তার stdout-কে ওই জায়গায় বসিয়ে দেয় গ) কমান্ডের exit code দেয় ঘ) ফাইল তৈরি করে
উত্তর: খ — `today=$(date +%F)` তারিখটি ভেরিয়েবলে রাখে। ক ভুল; গ ভুল, exit code `$?`-এ থাকে; ঘ ভুল।

D.9 অধ্যায় ৯ — Conditions, Loops and Control Flow (প্রশ্ন ৯.১–৯.৫)

৯.১ `[ -f "$file" ]` কী পরীক্ষা করে?
ক) directory আছে কি না খ) regular file আছে কি না গ) ফাইল খালি কি না ঘ) ফাইল executable কি না
উত্তর: খ — `-f` regular file পরীক্ষা করে। ক `-d`-এর কাজ; গ `-s` (non-empty) বা `! -s`-এর কাজ; ঘ `-x`-এর কাজ।

৯.২ সংখ্যা তুলনায় Bash-এ কোনটি সঠিক?
ক) `[ "$a" = "$b" ]` খ) `[ "$a" -eq "$b" ]` গ) `[ "$a" == "$b" ]` সংখ্যার জন্য ঘ) `[ "$a" > "$b" ]`
উত্তর: খ — সংখ্যার জন্য `-eq`, `-lt`, `-gt` ব্যবহার করতে হয়; `=` string তুলনা করে, তাই "10" ও "010" string হিসেবে ভিন্ন হতে পারে। ক string তুলনা, সংখ্যার জন্য ভুল পদ্ধতি; গ একই সমস্যা; ঘ `[ ]`-এ `>` redirection হয়ে যায়।

৯.৩ `case` statement কখন ব্যবহার করা ভালো?
ক) একটি শর্তে খ) একই মানের অনেকগুলো সম্ভাব্য মানের মধ্যে একটিকে বেছে নিতে, যেমন menu বা subcommand গ) loop-এ ঘ) ফাইল পড়তে
উত্তর: খ — case একাধিক pattern মেলাতে পরিষ্কার। ক ভুল, সেখানে if যথেষ্ট; গ ভুল; ঘ ভুল।

৯.৪ `for f in *.log` লুপে কোনো `.log` ফাইল না থাকলে কী হয়?
ক) লুপ চলে না খ) Bash-এ (nullglob ছাড়া) pattern-টি আক্ষরিক `*.log` হিসেবে একবার চলে — ফলে ভুল ফাইল নিয়ে কাজ হতে পারে গ) error হয় ঘ) সব ফাইল নিয়ে চলে
উত্তর: খ — এটাই Bash-এর default আচরণ; নিরাপদ পদ্ধতি হলো `shopt -s nullglob` বা ফাইল আছে কি না পরীক্ষা করা। ক ভুল, nullglob ছাড়া লুপ থামে না; গ ভুল, error হয় না; ঘ ভুল।

৯.৫ `while read -r line` লুপে `-r` কেন দেওয়া হয়?
ক) গতি বাড়াতে খ) backslash-কে escape হিসেবে ব্যাখ্যা না করে আক্ষরিক রাখতে গ) ফাইল মুছতে ঘ) লুপ থামাতে
উত্তর: খ — `-r` ছাড়া `\` পরের অক্ষরকে বিশেষভাবে ব্যাখ্যা করে, ফলে পাঠ বদলে যায়। ক ভুল; গ ভুল; ঘ ভুল।

D.10 অধ্যায় ১০ — Functions, Arguments and Script Structure (প্রশ্ন ১০.১–১০.৫)

১০.১ Bash function-এ `return 3` কী ফেরত দেয়?
ক) সংখ্যা 3 একটি মান হিসেবে খ) exit status 3 (0–255-এর মধ্যে), যা `$?`-এ পড়া যায় — function কোনো string বা বড় সংখ্যা return করতে পারে না গ) string "3" ঘ) কিছুই না
উত্তর: খ — Bash function-এ return শুধু status দেয়; ফলাফল দিতে stdout-এ print করতে হয় বা ভেরিয়েবলে রাখতে হয়। ক ভুল, কারণ এটি মান নয়; গ ভুল; ঘ ভুল।

১০.২ Function-এর ভেতরে `local count=0` কেন লেখা হয়?
ক) গতি বাড়াতে খ) ভেরিয়েবলটিকে শুধু ওই function-এর মধ্যে সীমাবদ্ধ রাখতে, যাতে বাইরের একই নামের ভেরিয়েবল বদলে না যায় গ) ভেরিয়েবল মুছতে ঘ) global করতে
উত্তর: খ — `local` ছাড়া assignment global হয়, ফলে নামের সংঘাত হয়। ক ভুল; গ ভুল; ঘ উল্টো।

১০.৩ Function-এর ভেতরে `$1`, `$@` কী বোঝায়?
ক) script-এর argument খ) ওই function-কে দেওয়া argument — function-এর নিজস্ব positional parameter গ) global ভেরিয়েবল ঘ) exit code
উত্তর: খ — function call হলে `$1` বদলে function-এর argument হয়। ক ভুল, script-এর argument function-এর বাইরে থাকে; গ ভুল; ঘ ভুল।

১০.৪ একটি reusable script-কে অন্য script থেকে ব্যবহারের নিরাপদ উপায় কোনটি?
ক) কপি-পেস্ট করা খ) `source lib.sh` (বা `. lib.sh`) দিয়ে function লোড করা, অথবা script-টিকে PATH-এ রেখে কমান্ড হিসেবে চালানো গ) script মুছে দেওয়া ঘ) সব কোড এক ফাইলে লেখা
উত্তর: খ — sourcing function শেয়ার করে, PATH-এ রাখা পুনর্ব্যবহার সহজ করে। ক রক্ষণাবেক্ষণ কঠিন করে; গ ভুল; ঘ ছোট কাজে চলে, বড় কাজে নয়।

১০.৫ Command-line argument না দিলে script-এর কী করা উচিত?
ক) চুপচাপ চলতে থাকা খ) ব্যবহারবিধি (usage) দেখিয়ে nonzero exit code-সহ থামা গ) অনুমান করে চালানো ঘ) ফাইল মুছে দেওয়া
উত্তর: খ — ভুল input-এ স্পষ্ট বার্তা দিয়ে থামা নিরাপদ ও পেশাদার আচরণ। ক ভুল, কারণ ভুল কাজ হতে পারে; গ বিপজ্জনক; ঘ ভুল।

D.11 অধ্যায় ১১ — Files, Text Processing and Command Pipelines (প্রশ্ন ১১.১–১১.৫)

১১.১ `cmd > out.log 2>&1` কী করে?
ক) শুধু stdout ফাইলে পাঠায় খ) stdout ও stderr — দুটোই `out.log`-এ পাঠায় গ) ফাইল মুছে দেয় ঘ) শুধু stderr দেখায়
উত্তর: খ — `2>&1` stderr-কে stdout-এর গন্তব্যে পাঠায়। ক ভুল, stderr বাদ পড়ত; গ ভুল; ঘ ভুল।

১১.২ `grep -r "ERROR" /var/log`-এ `-r` কী করে?
ক) ফাইল মুছে দেয় খ) directory-র ভেতরে recursiveভাবে সব ফাইলে খোঁজে গ) শুধু প্রথম ফাইলে খোঁজে ঘ) case-sensitive করে
উত্তর: খ — `-r` ছাড়া grep directory পড়তে পারে না। ক ভুল; গ ভুল; ঘ ভুল, case-insensitive করতে `-i` লাগে।

১১.৩ `awk '{print $2}'`-এ `$2` কী?
ক) দ্বিতীয় ফাইল খ) প্রতিটি লাইনের দ্বিতীয় field (ডিফল্ট whitespace দিয়ে ভাগ করা) গ) দ্বিতীয় লাইন ঘ) ভেরিয়েবল
উত্তর: খ — awk লাইনকে field-এ ভাগ করে; `$0` পুরো লাইন। ক ভুল; গ ভুল; ঘ ভুল।

১১.৪ ফাইলের নামে space থাকলে `find … | xargs rm` কেন বিপজ্জনক?
ক) ধীর হয় খ) xargs ডিফল্টে space দিয়ে নাম ভাঙে, ফলে একটি ফাইল ভেঙে একাধিক ভুল নামে পরিণত হতে পারে — নিরাপদ উপায় `find … -print0 | xargs -0` গ) ফাইল খুঁজে পায় না ঘ) কিছুই হয় না
উত্তর: খ — space-যুক্ত নামে ভুল ফাইল মোছা বা ভুল কমান্ড চলার ঝুঁকি থাকে। ক গৌণ; গ ভুল; ঘ ভুল।

১১.৫ `sort file.txt | uniq -c` কী করে?
ক) ফাইল মুছে দেয় খ) লাইনগুলো সাজিয়ে পাশাপাশি থাকা একই লাইনের সংখ্যা গোনে — `uniq` কাজ করে শুধু সাজানো (পাশাপাশি) লাইনে, তাই আগে sort লাগে গ) শুধু sort করে ঘ) লাইন সংখ্যা দেয়
উত্তর: খ — sort ছাড়া uniq দূরে থাকা একই লাইন আলাদা গোনে। ক ভুল; গ অসম্পূর্ণ; ঘ `wc -l`-এর কাজ।

D.12 অধ্যায় ১২ — Error Handling, Exit Codes and Debugging (প্রশ্ন ১২.১–১২.৫)

১২.১ Unix-এ exit code 0-এর অর্থ কী?
ক) ত্রুটি খ) সফল সমাপ্তি গ) ফাইল পাওয়া যায়নি ঘ) permission নেই
উত্তর: খ — 0 মানে সফল; nonzero মানে কোনো না কোনো ব্যর্থতা। ক উল্টো; গ সাধারণত 1 বা 2, কিন্তু নির্দিষ্ট নয়; ঘ সাধারণত 1 বা 126/127-এর মতো নির্দিষ্ট code, 0 নয়।

১২.২ `set -euo pipefail`-এর প্রতিটি অংশ কী করে?
ক) কিছুই করে না খ) `-e`: ব্যর্থ কমান্ডে থামে; `-u`: অনির্ধারিত ভেরিয়েবলে error দেয়; `pipefail`: pipeline-এর যেকোনো অংশ ব্যর্থ হলে pipeline-এর status ব্যর্থ হয় গ) শুধু -e কাজ করে ঘ) script দ্রুত করে
উত্তর: খ — তিনটি মিলে নিরাপদ script-এর ভিত্তি। ক ভুল; গ ভুল, তিনটিই কাজ করে; ঘ ভুল।

১২.৩ `trap 'cleanup' EXIT` কী করে?
ক) script মুছে দেয় খ) script যেভাবেই শেষ হোক (সফল বা ব্যর্থ), শেষে `cleanup` function চালায় — temporary ফাইল মোছার নির্ভরযোগ্য উপায় গ) signal বন্ধ করে ঘ) লুপ চালায়
উত্তর: খ — trap EXIT cleanup নিশ্চিত করে। ক ভুল; গ ভুল, trap signal ধরে, বন্ধ করে না; ঘ ভুল।

১২.৪ Idempotent script বলতে কী বোঝায়?
ক) একবারই চলে খ) একাধিকবার চালালেও একই ফল দেয় — দ্বিতীয়বার চালালে ক্ষতি বা দ্বিগুণ কাজ হয় না গ) দ্রুত চলে ঘ) root লাগে
উত্তর: খ — idempotency automation-এর মূল গুণ; যেমন directory থাকলে আবার তৈরি না করা। ক ভুল; গ ভুল; ঘ ভুল।

১২.৫ `command || { echo "failed" >&2; exit 1; }` প্যাটার্নের উদ্দেশ্য কী?
ক) কমান্ড লুকানো খ) কমান্ড ব্যর্থ হলে স্পষ্ট বার্তা stderr-এ দিয়ে nonzero code-সহ থামা গ) কমান্ড দুবার চালানো ঘ) ফাইল তৈরি করা
উত্তর: খ — এটি ব্যর্থতা স্পষ্ট করে ও চুপচাপ এগিয়ে যাওয়া ঠেকায়। ক ভুল; গ ভুল; ঘ ভুল।

D.13 অধ্যায় ১৩ — Advanced Bash Automation and Best Practices (প্রশ্ন ১৩.১–১৩.৫)

১৩.১ Temporary ফাইলের নিরাপদ উপায় কোনটি?
ক) `/tmp/myfile` লেখা খ) `mktemp` দিয়ে অনন্য নাম তৈরি করা এবং trap-এ মুছে ফেলা গ) একই নাম বারবার ব্যবহার করা ঘ) home-এ লেখা
উত্তর: খ — নির্দিষ্ট নামে লিখলে অন্য process বা আক্রমণকারী আগেই সেই নামে ফাইল/লিংক বানিয়ে রাখতে পারে। ক অনিরাপদ; গ সংঘাত তৈরি করে; ঘ নিরাপত্তা দেয় না।

১৩.২ Lock file কেন ব্যবহার করা হয়?
ক) ফাইল লুকাতে খ) একই script-এর দুটি copy একসঙ্গে চলা ঠেকাতে — দ্বিতীয় copy lock না পেলে থামে গ) গতি

বাড়াতে ঘ) log লিখতে
উত্তর: খ — cron-এ একই job ওভারল্যাপ করলে তথ্য নষ্ট হতে পারে; `flock` এই কাজের মানসম্মত উপায়। ক ভুল; গ ভুল; ঘ ভুল।

১৩.৩ ShellCheck কী?
ক) shell-এর গতি মাপার tool খ) Bash script-এর সাধারণ ভুল, quoting সমস্যা ও বিপজ্জনক প্যাটার্ন ধরার static analysis tool গ) ফাইল মুছে ফেলার tool ঘ) editor
উত্তর: খ — ShellCheck চালানোর আগেই ভুল ধরে। ক ভুল; গ ভুল; ঘ ভুল।

১৩.৪ Command injection-এর ঝুঁকি কোথায় সবচেয়ে বেশি?
ক) নির্দিষ্ট কমান্ডে খ) ব্যবহারকারীর দেওয়া input-কে যাচাই ছাড়া shell কমান্ডে বসালে — আক্রমণকারী `; rm -rf`-এর মতো অংশ ঢুকিয়ে দিতে পারে গ) ফাইল পড়ায় ঘ) log লেখায়
উত্তর: খ — untrusted input কখনো shell-এ সরাসরি বসানো যায় না; array বা Python-এর `shell=False` ব্যবহার করতে হয়। ক নিরাপদ; গ ও ঘ নিজে থেকে injection নয়।

১৩.৫ Portability-র জন্য Bash script-এ কী এড়ানো উচিত?
ক) `#!/bin/bash` খ) শুধু একটি distro-তে থাকা বিশেষ tool বা Bash-এর নতুন বৈশিষ্ট্যের উপর নির্ভর করা, যখন POSIX বা সাধারণ উপায়ে কাজটি করা যায় গ) function ব্যবহার ঘ) `set -e`
উত্তর: খ — portability মানে বিভিন্ন পরিবেশে চলা। ক ভুল, shebang স্পষ্টতা দেয়; গ ভুল; ঘ ভুল, `set -e` নিরাপত্তা দেয়।

D.14 অধ্যায় ১৪ — Python Fundamentals for Automation (প্রশ্ন ১৪.১–১৪.৫)

১৪.১ Python-এ virtual environment (venv) কেন ব্যবহার করা হয়?
ক) গতি বাড়াতে খ) প্রতিটি প্রজেক্টের dependency আলাদা রাখতে, যাতে এক প্রজেক্টের library অন্য প্রজেক্টের সঙ্গে সংঘাত না করে গ) ফাইল লুকাতে ঘ) Python install করতে
উত্তর: খ — venv প্রজেক্ট-ভিত্তিক বিচ্ছিন্নতা দেয়। ক ভুল; গ ভুল; ঘ ভুল, venv Python install করে না।

১৪.২ Python-এ `list` ও `tuple`-এর মূল পার্থক্য কী?
ক) কোনো পার্থক্য নেই খ) list পরিবর্তনযোগ্য (mutable), tuple অপরিবর্তনীয় (immutable) গ) tuple দ্রুত নয় ঘ) list-এ সংখ্যা রাখা যায় না
উত্তর: খ — tuple একবার তৈরি হলে বদলানো যায় না, তাই স্থির তথ্যের জন্য নিরাপদ। ক ভুল; গ ভুল, tuple সাধারণত হালকা; ঘ ভুল।

১৪.৩ Dictionary-তে key খুঁজে না পেলে `d["missing"]` কী করে?
ক) None দেয় খ) `KeyError` exception তোলে গ) খালি string দেয় ঘ) 0 দেয়
উত্তর: খ — সরাসরি indexing না থাকলে KeyError তোলে; নিরাপদে `d.get("missing")` None দেয়। ক `.get()`-এর আচরণ; গ ও ঘ ভুল।

১৪.৪ `if __name__ == "__main__":` কেন লেখা হয়?
ক) গতি বাড়াতে খ) ফাইলটি সরাসরি চালালে main কোড চলবে, কিন্তু import করলে চলবে না — module হিসেবে পুনর্ব্যবহারের জন্য গ) ফাইল লুকাতে ঘ) error ধরতে
উত্তর: খ — এটি import ও execution আলাদা করে। ক ভুল; গ ভুল; ঘ ভুল, error ধরা try/except-এর কাজ।

১৪.৫ `requirements.txt`-এর কাজ কী?
ক) Python install করা খ) প্রজেক্টের dependency ও তাদের সংস্করণ তালিকাভুক্ত করা, যাতে `pip install -r`

`requirements.txt` দিয়ে একই পরিবেশ আবার তৈরি করা যায় গ) কোড লেখা ঘ) test চালানো
উত্তর: খ — এটি পুনরুৎপাদনযোগ্য পরিবেশের ভিত্তি। ক ভুল; গ ভুল; ঘ ভুল, test চালায় pytest।

D.15 অধ্যায় ১৫ — Files, Directories and System Operations (প্রশ্ন ১৫.১–১৫.৫)

১৫.১ `pathlib.Path` কেন `os.path`-এর চেয়ে পছন্দ করা হয়?
ক) দ্রুত খ) Path object-ভিত্তিক, পড়তে সহজ ও OS-নিরপেক্ষ — `path / "file.txt"` দিয়ে যুক্ত করা যায় গ) শুধু Linux-এ চলে ঘ) ফাইল মুছে দেয়
উত্তর: খ — pathlib string জোড়া দেওয়ার ভুল কমায়। ক গৌণ; গ ভুল, pathlib cross-platform; ঘ ভুল।

১৫.২ `with open("a.txt") as f:` ব্যবহারের সুবিধা কী?
ক) ফাইল দ্রুত খোলে খ) block শেষ হলে ফাইল স্বয়ংক্রিয়ভাবে বন্ধ হয় — exception হলেও গ) ফাইল মুছে দেয় ঘ) ফাইল বড় করে
উত্তর: খ — with statement resource বন্ধ করা নিশ্চিত করে। ক গৌণ; গ ভুল; ঘ ভুল।

১৫.৩ `shutil.copy2()` ও `shutil.copy()`-এর পার্থক্য কী?
ক) একই খ) copy2 ফাইলের metadata (সময়, permission) সংরক্ষণের চেষ্টা করে, copy শুধু বিষয়বস্তু কপি করে গ) copy2 ফোল্ডার কপি করে ঘ) copy ফাইল মুছে দেয়
উত্তর: খ — backup-এ metadata গুরুত্বপূর্ণ বলে copy2 পছন্দ করা হয়। ক ভুল; গ ভুল, ফোল্ডারের জন্য `copytree()`; ঘ ভুল।

১৫.৪ Backup-এর আগে ফাইলের checksum (যেমন SHA-256) কেন নেওয়া হয়?
ক) ফাইল ছোট করতে খ) পরে backup-এর অখণ্ডতা যাচাই করতে — কপি করার সময় তথ্য নষ্ট হয়েছে কি না, তা ধরা যায় গ) ফাইল encrypt করতে ঘ) ফাইল মুছতে
উত্তর: খ — checksum মিললে কপিটি বিশ্বাসযোগ্য। ক ভুল; গ ভুল, checksum encryption নয়; ঘ ভুল।

১৫.৫ `Path("dir").mkdir(parents=True, exist_ok=True)` কী করে?
ক) ফাইল তৈরি করে খ) প্রয়োজনে parent directory-সহ directory তৈরি করে; আগে থেকে থাকলে error দেয় না গ) directory মুছে দেয় ঘ) permission বদলায়
উত্তর: খ — `parents=True` মাঝখানের directory বানায়, `exist_ok=True` আগে থাকলে শান্ত থাকে। ক ভুল; গ `rmdir`/`shutil.rmtree`-এর কাজ; ঘ `chmod`-এর কাজ।

D.16 অধ্যায় ১৬ — Processes, Subprocesses and Command Execution (প্রশ্ন ১৬.১–১৬.৫)

১৬.১ `subprocess.run(["ls", "-l"], capture_output=True, text=True)`-এ তালিকা (list) ব্যবহারের কারণ কী?
ক) গতি বাড়াতে খ) shell ছাড়া সরাসরি program চালানো — এতে shell injection-এর ঝুঁকি থাকে না গ) ফাইল তৈরি করতে ঘ) output লুকাতে
উত্তর: খ — list আকারে argument দিলে shell ব্যাখ্যা করে না; `shell=True` ও string কমান্ড injection-এর দরজা খোলে। ক গৌণ; গ ভুল; ঘ ভুল, capture_output output ধরে রাখে।

১৬.২ `subprocess.run(..., check=True)` কী করে?
ক) কমান্ড দুবার চালায় খ) কমান্ড nonzero exit code দিলে `CalledProcessError` exception তোলে গ) output

দেখায় ঘ) timeout দেয়
উত্তর: খ — check=True ব্যর্থতা চুপচাপ পেরিয়ে যাওয়া ঠেকায়। ক ভুল; গ capture_output-এর কাজ; ঘ timeout parameter-এর কাজ।

১৬.৩ `subprocess.run(..., timeout=10)`-এ timeout হলে কী হয়?
ক) কমান্ড চলতেই থাকে খ) `TimeoutExpired` exception ওঠে এবং process-টি বন্ধ করা হয় গ) 0 return করে ঘ) ফাইল তৈরি হয়
উত্তর: খ — timeout ঝুলে থাকা automation ঠেকায়। ক ভুল; গ ভুল; ঘ ভুল।

১৬.৪ stdout ও stderr আলাদা রাখার সুবিধা কী?
ক) কোনো সুবিধা নেই খ) আসল ফলাফল (stdout) ও ত্রুটি/সতর্কবার্তা (stderr) আলাদা করে বিশ্লেষণ ও log করা যায় গ) গতি বাড়ে ঘ) ফাইল ছোট হয়
উত্তর: খ — আলাদা stream-এ ভুল বার্তা ফলাফলের সঙ্গে মিশে যায় না। ক ভুল; গ ভুল; ঘ ভুল।

১৬.৫ ব্যবহারকারীর দেওয়া ফাইলের নাম দিয়ে কমান্ড চালাতে নিরাপদ উপায় কোনটি?
ক) `shell=True` দিয়ে string বানানো খ) argument list-এ নামটি আলাদা element হিসেবে দেওয়া (`shell=False`) গ) নামটি quote না করা ঘ) `os.system()` ব্যবহার করা
উত্তর: খ — list-এ argument দিলে `; rm -rf`-এর মতো অংশও আক্ষরিক নাম হিসেবে বিবেচিত হয়। ক ও ঘ shell injection-এর ঝুঁকিপূর্ণ; গ ভুল।

D.17 অধ্যায় ১৭ — APIs, HTTP Requests and JSON Processing (প্রশ্ন ১৭.১–১৭.৫)

১৭.১ HTTP GET ও POST-এর মূল পার্থক্য কী?
ক) একই কাজ খ) GET তথ্য পড়তে (safe, idempotent), POST নতুন resource তৈরি বা কাজ করতে তথ্য পাঠায় গ) GET তথ্য পাঠায় ঘ) POST তথ্য পড়ে
উত্তর: খ — GET-এর body থাকে না বললেই চলে; POST body-তে তথ্য পাঠায়। ক ভুল; গ ও ঘ উল্টো।

১৭.২ HTTP status code 404 ও 500-এর অর্থ কী?
ক) দুটোই সফল খ) 404 = resource পাওয়া যায়নি (client-এর ভুল), 500 = সার্ভারের ভেতরের ত্রুটি গ) 404 = সার্ভার বন্ধ ঘ) 500 = পাওয়া যায়নি
উত্তর: খ — 4xx client-এর দিকের ভুল, 5xx সার্ভারের দিকের ভুল। ক ভুল; গ ভুল; ঘ উল্টো।

১৭.৩ API token কোথায় রাখা উচিত?
ক) source code-এ খ) environment variable বা secret manager-এ, কখনো Git-এ commit করা যাবে না গ) README-তে ঘ) public ফাইলে
উত্তর: খ — code-এ token থাকলে repository ফাঁস হলেই token ফাঁস হয়। ক, গ ও ঘ সবই নিরাপত্তা লঙ্ঘন।

১৭.৪ API call-এ timeout দেওয়া কেন জরুরি?
ক) গতি বাড়াতে খ) সার্ভার সাড়া না দিলে automation অনির্দিষ্টকাল ঝুলে না থাকে — নির্দিষ্ট সময় পর ব্যর্থ হয়ে retry বা log করা যায় গ) token বাঁচাতে ঘ) JSON ছোট করতে
উত্তর: খ — timeout ছাড়া একটি ঝুলে থাকা request পুরো job আটকে দেয়। ক গৌণ; গ ভুল; ঘ ভুল।

১৭.৫ Rate limiting-এ 429 status পেলে সঠিক আচরণ কী?
ক) সঙ্গে সঙ্গে আবার চেষ্টা করা খ) `Retry-After` header দেখে অপেক্ষা করে ধীরে আবার চেষ্টা করা (backoff), বারবার আঘাত না করা গ) token বদলানো ঘ) request মুছে দেওয়া

উত্তর: খ — 429 মানে "খুব বেশি অনুরোধ" — অবিলম্বে retry করলে আরও block হওয়ার ঝুঁকি থাকে। ক ভুল; গ ভুল; ঘ ভুল।

D.18 অধ্যায় ১৮ — YAML, Configuration and Environment Management (প্রশ্ন ১৮.১–১৮.৫)

১৮.১ JSON ও YAML-এর মূল পার্থক্য কী?
ক) একই জিনিস খ) JSON কঠোর bracket/quote-ভিত্তিক; YAML indentation-ভিত্তিক, মানুষের পড়ার জন্য সহজ এবং comment সমর্থন করে গ) YAML শুধু Python-এ চলে ঘ) JSON-এ comment লেখা যায়
উত্তর: খ — YAML config-এর জন্য জনপ্রিয়, JSON API-র জন্য। ক ভুল; গ ভুল, YAML ভাষা-নিরপেক্ষ; ঘ ভুল, JSON-এ comment নেই।

১৮.২ YAML-এ indentation ভুল হলে কী হয়?
ক) কিছু হয় না খ) parser ভুল গঠন বোঝে বা error দেয় — YAML-এ tab ব্যবহার করা যায় না, শুধু space গ) ফাইল মুছে যায় ঘ) মান বদলে যায় না
উত্তর: খ — indentation-ই YAML-এর গঠন নির্ধারণ করে। ক ভুল; গ ভুল; ঘ ভুল।

১৮.৩ `.env` ফাইল Git-এ commit করা উচিত কি?
ক) হ্যাঁ, সবসময় খ) না — এতে secret থাকে; শুধু `.env.example` (খালি মানসহ নমুনা) commit করা যায় গ) শুধু production-এ ঘ) কখনো ফাইল বানানো যাবে না
উত্তর: খ — .env-এ API key/password থাকে, তাই `.gitignore`-এ রাখতে হয়। ক বিপজ্জনক; গ ভুল; ঘ ভুল, .env ব্যবহার করা যায়, শুধু commit করা যায় না।

১৮.৪ Configuration validation কেন করা হয়?
ক) গতি বাড়াতে খ) প্রোগ্রাম শুরুতেই ভুল বা অনুপস্থিত config ধরে স্পষ্ট বার্তা দিতে, যাতে আধা-চলা অবস্থায় ব্যর্থ না হয় গ) ফাইল ছোট করতে ঘ) log লিখতে
উত্তর: খ — fail-fast নীতি: শুরুতে ভুল ধরা পড়লে debugging সহজ হয়। ক ভুল; গ ভুল; ঘ ভুল।

১৮.৫ Configuration ও code আলাদা রাখার নীতি কী?
ক) একই ফাইলে রাখা খ) পরিবেশ-ভেদে বদলায় এমন মান (URL, পথ, সীমা) code-এর বাইরে config/env-এ রাখা, যাতে code না বদলে পরিবেশ বদলানো যায় গ) config মুছে দেওয়া ঘ) hardcode করা
উত্তর: খ — এতে একই code dev, staging ও production-এ চলে। ক ও ঘ এই নীতির বিপরীত; গ ভুল।

D.19 অধ্যায় ১৯ — Logging, Exceptions and Reliable Automation (প্রশ্ন ১৯.১–১৯.৫)

১৯.১ Python logging-এ level-গুলোর সঠিক ক্রম কোনটি?
ক) ERROR > CRITICAL খ) DEBUG < INFO < WARNING < ERROR < CRITICAL গ) INFO < DEBUG ঘ) WARNING < INFO
উত্তর: খ — DEBUG সবচেয়ে বিস্তারিত, CRITICAL সবচেয়ে গুরুতর। ক ভুল, CRITICAL সবচেয়ে উঁচু; গ ভুল; ঘ ভুল।

১৯.২ `print()`-এর বদলে logging module ব্যবহারের প্রধান সুবিধা কী?
ক) গতি খ) level, সময়, ফাইলে লেখা ও ঘূর্ণন (rotation) নিয়ন্ত্রণ করা যায় — production-এ print বিশৃঙ্খল হয় গ) রঙ ঘ) কম কোড
উত্তর: খ — logging কাঠামোবদ্ধ ও নিয়ন্ত্রণযোগ্য। ক গৌণ; গ ভুল; ঘ ভুল।

১৯.৩ `except Exception:` দিয়ে সব exception ধরে চুপচাপ এগিয়ে যাওয়া কেন খারাপ?
ক) খারাপ নয় খ) আসল ত্রুটি লুকিয়ে যায়, debugging কঠিন হয় এবং ভুল অবস্থায় কাজ এগিয়ে যায় গ) গতি কমে ঘ) ফাইল বড় হয়
উত্তর: খ — শুধু সামলানো যায় এমন নির্দিষ্ট exception ধরতে হয় এবং log করতে হয়। ক ভুল; গ ভুল; ঘ ভুল।

১৯.৪ Retry-তে exponential backoff কী?
ক) সঙ্গে সঙ্গে retry খ) প্রতিবার ব্যর্থতার পর অপেক্ষার সময় বাড়িয়ে দেওয়া (যেমন 1s, 2s, 4s), যাতে সার্ভারের উপর চাপ না পড়ে গ) একবারই চেষ্টা ঘ) token বদলানো
উত্তর: খ — backoff সার্ভারকে সুস্থ হওয়ার সময় দেয়। ক ভুল, তা সার্ভারকে আরও চাপ দেয়; গ ভুল; ঘ ভুল।

১৯.৫ `RotatingFileHandler` কী করে?
ক) ফাইল মুছে দেয় খ) log ফাইল নির্দিষ্ট আকারে পৌঁছালে নতুন ফাইলে লেখা শুরু করে ও পুরনো ফাইল সংরক্ষণ করে, ফলে disk ভরে যায় না গ) log বন্ধ করে ঘ) log encrypt করে
উত্তর: খ — এতে log সীমিত জায়গায় থাকে। ক ভুল; গ ভুল; ঘ ভুল।

D.20 অধ্যায় ২০ — Scheduling, Cron and Background Automation (প্রশ্ন ২০.১–২০.৫)

২০.১ Cron expression-এর field-গুলোর সঠিক ক্রম কোনটি?
ক) hour minute day month weekday খ) minute hour day-of-month month day-of-week গ) second minute hour day month ঘ) day month year hour minute
উত্তর: খ — cron-এ পাঁচটি field: মিনিট, ঘণ্টা, মাসের দিন, মাস, সপ্তাহের দিন। ক ক্রম উল্টো; গ ভুল, cron-এ second field নেই; ঘ ভুল।

২০.২ `*/15 * * * *` মানে কী?
ক) প্রতি 15 ঘণ্টায় খ) প্রতি 15 মিনিটে গ) দিনে 15 বার ঘ) মাসে 15 তারিখে
উত্তর: খ — প্রথম field মিনিট, `*/15` মানে প্রতি 15 মিনিটে। ক ভুল, ঘণ্টা দ্বিতীয় field; গ ভুল; ঘ ভুল।

২০.৩ Cron job-এ environment variable (যেমন PATH) নিয়ে সমস্যা কেন হয়?
ক) সমস্যা হয় না খ) cron ন্যূনতম environment নিয়ে চলে — interactive shell-এর PATH, ভেরিয়েবল বা alias থাকে না, তাই পূর্ণ পথ (absolute path) ব্যবহার করতে হয় গ) cron PATH মুছে দেয় ঘ) cron root-এ চলে
উত্তর: খ — terminal-এ চলা script cron-এ ব্যর্থ হওয়ার এটাই সাধারণ কারণ। ক ভুল; গ ভুল; ঘ ভুল, cron ব্যবহারকারীর crontab অনুযায়ী চলে।

২০.৪ Cron job-এর output কোথায় যায়, যদি redirect না করা হয়?
ক) হারিয়ে যায় খ) ব্যবহারকারীর local mail-এ পাঠানো হয় (বা systemd journal-এ, ব্যবস্থা অনুযায়ী) গ) পর্দায় দেখায় ঘ) ফাইলে লেখে
উত্তর: খ — তাই ভালো অভ্যাস হলো stdout/stderr স্পষ্টভাবে log ফাইলে redirect করা। ক ভুল, mail-এ যায়; গ ভুল, cron-এর পর্দা নেই; ঘ ভুল, নিজে থেকে ফাইলে লেখে না।

২০.৫ একই cron job দুবার একসঙ্গে চলা ঠেকাতে কী করা হয়?
ক) কিছুই করা যায় না খ) `flock` দিয়ে lock নেওয়া, যাতে আগের copy চলতে থাকলে নতুন copy থামে গ) job মুছে দেওয়া ঘ) সময় বদলানো
উত্তর: খ — flock একসঙ্গে চলা ঠেকায়। ক ভুল; গ ভুল; ঘ ওভারল্যাপ কমাতে পারে, কিন্তু নিশ্চিত করে না।

D.21 অধ্যায় ২১ — SSH Automation and Remote System Management (প্রশ্ন ২১.১–২১.৫)

২১.১ SSH key pair-এ public ও private key-এর ভূমিকা কী?
ক) দুটোই শেয়ার করা যায় খ) public key সার্ভারে থাকে, private key শুধু নিজের কম্পিউটারে থাকে — private key কখনো শেয়ার করা যাবে না গ) private key সার্ভারে থাকে ঘ) দুটোই password
উত্তর: খ — private key দিয়ে পরিচয় প্রমাণিত হয়; তা ফাঁস হলে যে কেউ সার্ভারে ঢুকতে পারে। ক বিপজ্জনক; গ উল্টো; ঘ ভুল।

২১.২ `known_hosts`-এ সার্ভারের host key থাকার উদ্দেশ্য কী?
ক) গতি বাড়ানো খ) পরের সংযোগে সার্ভারটি আগের সার্ভারই কি না, তা যাচাই করা — মাঝখানে কেউ (man-in-the-middle) ঢুকলে সতর্ক করা গ) password মনে রাখা ঘ) ফাইল শেয়ার করা
উত্তর: খ — host key বদলে গেলে SSH সতর্ক করে; যাচাই ছাড়া এগিয়ে যাওয়া বিপজ্জনক। ক ভুল; গ ভুল; ঘ ভুল।

২১.৩ `scp` ও `rsync`-এর মধ্যে rsync-এর বড় সুবিধা কী?
ক) rsync দ্রুত নয় খ) rsync শুধু বদলে যাওয়া অংশ কপি করে, ফলে বড় ফাইল/ফোল্ডার বারবার কপিতে সময় ও ব্যান্ডউইথ বাঁচে গ) scp encrypt করে না ঘ) rsync password চায় না
উত্তর: খ — rsync incremental কপি করে। ক ভুল; গ ভুল, দুটোই SSH দিয়ে encrypt হয়; ঘ ভুল, দুটোতেই authentication লাগে।

২১.৪ `ssh user@host "df -h"` কী করে?
ক) সার্ভারে login করে interactive shell খোলে খ) সার্ভারে সংযোগ করে `df -h` কমান্ড চালিয়ে তার output ফিরিয়ে দেয়, তারপর সংযোগ বন্ধ করে গ) ফাইল কপি করে ঘ) সার্ভার বন্ধ করে
উত্তর: খ — কমান্ড দিলে SSH remote-এ তা চালিয়ে ফল ফেরত দেয়। ক ভুল, কমান্ড ছাড়া interactive shell খোলে; গ scp/rsync-এর কাজ; ঘ ভুল।

২১.৫ Private key ফাইলের permission কেমন হওয়া উচিত?
ক) 644 খ) 600 — শুধু মালিক পড়তে পারবে; বেশি খোলা permission থাকলে SSH key ব্যবহার করতে অস্বীকার করে গ) 777 ঘ) 755
উত্তর: খ — SSH নিরাপত্তার জন্য private key-এর permission কঠোরভাবে পরীক্ষা করে। ক, গ ও ঘ — সবই বেশি খোলা।

D.22 অধ্যায় ২২ — Building Reusable Automation Tools (প্রশ্ন ২২.১–২২.৫)

২২.১ CLI tool-এ `argparse` ব্যবহারের সুবিধা কী?
ক) গতি বাড়ে খ) argument parsing, সাহায্য বার্তা (`--help`), type যাচাই ও ভুল বার্তা স্বয়ংক্রিয়ভাবে পাওয়া যায় গ) ফাইল তৈরি হয় ঘ) log লেখে
উত্তর: খ — argparse হাতে parsing লেখার ভুল কমায়। ক গৌণ; গ ভুল; ঘ ভুল।

২২.২ Dry-run mode কী?
ক) পরীক্ষা ছাড়া চালানো খ) আসল পরিবর্তন না করে কী করা হতো, তা দেখানো — নিরাপদে যাচাই করার উপায় গ) ফাইল মুছে দেওয়া ঘ) দ্রুত চালানো
উত্তর: খ — dry-run দিয়ে production-এ চালানোর আগে প্রভাব দেখা যায়। ক উল্টো; গ ভুল; ঘ ভুল।

২২.৩ একটি automation tool-কে modular করার সুবিধা কী?
ক) ফাইল বেশি হয় খ) প্রতিটি অংশ আলাদাভাবে পরীক্ষা, পুনর্ব্যবহার ও বদলানো যায় — একটি অংশের ভুল পুরো tool নষ্ট করে না গ) গতি কমে ঘ) কোড লুকানো যায়
উত্তর: খ — modular কাঠামো রক্ষণাবেক্ষণ সহজ করে। ক ভুল, ফাইল বেশি হওয়া উদ্দেশ্য নয়; গ ভুল; ঘ ভুল।

২২.৪ Tool-এর প্রথম ধাপে configuration যাচাই (validate) করা কেন ভালো?
ক) সময় নষ্ট খ) ভুল config নিয়ে আধা কাজ করে তথ্য নষ্ট করার আগেই স্পষ্ট বার্তা দিয়ে থামা যায় (fail-fast) গ) log বড় হয় ঘ) token বাঁচে
উত্তর: খ — শুরুতে যাচাই করলে ব্যর্থতা আগেই ধরা পড়ে। ক ভুল; গ ভুল; ঘ ভুল।

২২.৫ Idempotent backup tool-এ দ্বিতীয়বার একই backup চালালে কী হওয়া উচিত?
ক) দ্বিগুণ ফাইল তৈরি হওয়া খ) একই ফল — আগের backup মুছে না গিয়ে বা নষ্ট না হয়ে, প্রয়োজনে নতুন সংস্করণ তৈরি হওয়া, কিন্তু ভুল অবস্থা তৈরি না হওয়া গ) পুরনো backup মুছে যাওয়া ঘ) error ছাড়া কিছু না হওয়া
উত্তর: খ — idempotency মানে পুনরায় চালানো নিরাপদ। ক ভুল; গ ভুল; ঘ ভুল, নতুন backup তৈরি হওয়া স্বাভাবিক হতে পারে, কিন্তু অবস্থা সামঞ্জস্যপূর্ণ থাকতে হবে।

D.23 অধ্যায় ২৩ — Code Quality, Testing and Documentation (প্রশ্ন ২৩.১–২৩.৫)

২৩.১ pytest-এ test function-এর নাম কেমন হতে হয়?
ক) যেকোনো নাম খ) `test_` দিয়ে শুরু হতে হয়, যাতে pytest তা খুঁজে পায় গ) `check_` দিয়ে ঘ) বড় হাতের অক্ষরে
উত্তর: খ — pytest ডিফল্টে `test_*.py` ফাইল ও `test_*` function খোঁজে। ক ভুল, নাম না মিললে test চলবে না; গ ও ঘ ভুল।

২৩.২ Mocking কেন ব্যবহার করা হয়?
ক) গতি বাড়াতে খ) বাইরের জিনিস (API, কমান্ড, ফাইল সিস্টেম) ছাড়া test চালাতে — আসল সার্ভারে আঘাত না করে আচরণ যাচাই করা যায় গ) কোড লুকাতে ঘ) test মুছতে
উত্তর: খ — mock test-কে দ্রুত, নির্ভরযোগ্য ও নিরাপদ করে। ক গৌণ সুবিধা, মূল কারণ নয়; গ ভুল; ঘ ভুল।

২৩.৩ ShellCheck ও Python linting (যেমন ruff) চালানোর উদ্দেশ্য কী?
ক) কোড দ্রুত করা খ) চালানোর আগেই সাধারণ ভুল, শৈলীর সমস্যা ও বিপজ্জনক প্যাটার্ন ধরা গ) ফাইল compress করা ঘ) test লেখা
উত্তর: খ — linter static analysis করে। ক গৌণ; গ ভুল; ঘ ভুল।

২৩.৪ ভালো README-তে কী থাকা উচিত?
ক) শুধু নাম খ) উদ্দেশ্য, install ধাপ, ব্যবহারবিধি, উদাহরণ, config ও troubleshooting গ) password ঘ) সব কোড
উত্তর: খ — README ব্যবহারকারীর প্রথম নির্দেশিকা। ক অসম্পূর্ণ; গ নিরাপত্তা লঙ্ঘন; ঘ ভুল, কোড ফাইলে থাকে।

২৩.৫ Integration test ও unit test-এর পার্থক্য কী?
ক) একই জিনিস খ) unit test একটি ছোট অংশ একা পরীক্ষা করে; integration test একাধিক অংশ একসঙ্গে কাজ করে কি না, তা পরীক্ষা করে গ) unit test ধীর ঘ) integration test কোড পড়ে না
উত্তর: খ — দুটি আলাদা স্তরের পরীক্ষা। ক ভুল; গ উল্টো, unit test সাধারণত দ্রুত; ঘ ভুল।

D.24 অধ্যায় ২৪ — Automation Architecture and Production Practices (প্রশ্ন ২৪.১–২৪.৫)

২৪.১ একটি সহজ কাজের জন্য Bash ও জটিল তথ্য প্রক্রিয়ার জন্য Python — এই পছন্দের যুক্তি কী?
ক) Bash সবসময় ভালো খ) Bash কমান্ড একত্র করতে দ্রুত ও সহজ; Python জটিল তথ্য, API ও error handling-এ শক্তিশালী ও পড়তে সহজ গ) Python সবসময় ভালো ঘ) ভাষা কোনো ব্যাপার নয়
উত্তর: খ — সঠিক tool বেছে নেওয়া engineering সিদ্ধান্ত। ক ও গ একপেশে; ঘ ভুল, ভাষার শক্তি কাজের সঙ্গে মেলাতে হয়।

২৪.২ Production automation-এ observability বলতে কী বোঝায়?
ক) শুধু log লেখা খ) log, metric ও status-এর মাধ্যমে automation কী করছে, কোথায় ব্যর্থ হচ্ছে, তা দেখা ও বোঝা যায় গ) পর্দায় দেখা ঘ) ফাইল গোনা
উত্তর: খ — observability ছাড়া ব্যর্থতা অন্ধকারে থাকে। ক অসম্পূর্ণ, log তার একটি অংশ; গ ভুল; ঘ ভুল।

২৪.৩ Production-এ secret (API key, password) রাখার নিরাপদ উপায় কোনটি?
ক) code-এ লেখা খ) environment variable বা secret manager-এ, code ও Git-এর বাইরে গ) README-তে ঘ) chat-এ পাঠানো
উত্তর: খ — secret কখনো code/repository-তে রাখা যায় না। ক, গ ও ঘ — সবই ফাঁসের পথ।

২৪.৪ Automation ব্যর্থ হলে সবচেয়ে নিরাপদ আচরণ কোনটি?
ক) চুপচাপ এগিয়ে যাওয়া খ) স্পষ্টভাবে log করে, nonzero exit code দিয়ে থামা ও আংশিক পরিবর্তন না রাখা (বা rollback করা) গ) আবার চেষ্টা করা ঘ) ফাইল মুছে দেওয়া
উত্তর: খ — নিরাপদ ব্যর্থতা (safe failure) তথ্য নষ্ট হওয়া ঠেকায়। ক বিপজ্জনক; গ সবসময় নিরাপদ নয়; ঘ ভুল।

২৪.৫ একটি automation tool production-এ দেওয়ার আগে কী যাচাই করা উচিত?
ক) শুধু নিজের কম্পিউটারে চললেই হবে খ) test, dry-run, config validation, log, error handling, idempotency ও rollback পরিকল্পনা — সব যাচাই করা গ) কোড লম্বা কি না ঘ) নাম সুন্দর কি না
উত্তর: খ — production-এ যাওয়ার আগে এই তালিকা সম্পূর্ণ করতে হয়। ক ভুল, পরিবেশ ভিন্ন হয়; গ ও ঘ তুচ্ছ।

> **PRO TIP:** ১২০টি প্রশ্নের মধ্যে কোনো অধ্যায়ে বারবার ভুল হলে সেই অধ্যায়ের সংশ্লিষ্ট lab আবার করুন — শুধু ব্যাখ্যা পড়ে নয়, হাতে চালিয়ে শিখুন।

D.25 Short-Answer Questions — সংক্ষিপ্ত প্রশ্ন ও মডেল উত্তর

১. Git-এ snapshot বলতে কী বোঝায়? Delta-ভিত্তিক ব্যবস্থা থেকে এটি কীভাবে আলাদা?
মডেল উত্তর: Git প্রতিটি commit-এ পুরো প্রজেক্টের snapshot সংরক্ষণ করে — প্রতিটি ফাইলের বিষয়বস্তুর hash (blob) ও directory কাঠামো (tree)। ফাইল না বদলালে আগের blob-এর দিকে pointer দেওয়া হয়, তাই জায়গা নষ্ট হয় না। Delta-ভিত্তিক ব্যবস্থা (যেমন পুরনো CVS) শুধু পরিবর্তনের পার্থক্য রাখে, ফলে কোনো সংস্করণ পেতে সব পার্থক্য একত্র করতে হয়। Git-এ যেকোনো commit সরাসরি তার snapshot থেকে পাওয়া যায়, তাই checkout ও ইতিহাস দেখা দ্রুত হয়।

২. `git rebase` কখন ব্যবহার করা নিরাপদ, আর কখন বিপজ্জনক?
মডেল উত্তর: নিজের local, এখনো push করা হয়নি এমন branch-এ rebase নিরাপদ — এতে ইতিহাস পরিষ্কার হয়। অন্যরা ব্যবহার করছে এমন shared/public branch-এ rebase বিপজ্জনক, কারণ এটি commit-এর hash বদলে দেয়; অন্যদের local ইতিহাসের সঙ্গে সংঘাত হয়। Shared branch-এ merge ব্যবহার করতে হয়।

৩. Bash-এ `set -euo pipefail` কেন প্রায় সব production script-এর শুরুতে লেখা হয়?
মডেল উত্তর: `-e` ব্যর্থ কমান্ডে script থামায়, `-u` অনির্ধারিত ভেরিয়েবল ব্যবহারে error দেয়, `pipefail` pipeline-এর যেকোনো অংশ ব্যর্থ হলে পুরো pipeline-কে ব্যর্থ ধরে। তিনটি মিলে ভুল চুপচাপ এগিয়ে যাওয়া ঠেকায়, ফলে ব্যর্থতা আগেই ও স্পষ্টভাবে ধরা পড়ে।

৪. Bash-এ unquoted ভেরিয়েবল (`$file`) কেন বিপজ্জনক? উদাহরণ দিন।
মডেল উত্তর: unquoted expansion-এ word splitting ও globbing হয়। যেমন ফাইলের নাম `my report.txt` হলে `rm $file` দুটি নাম — `my` ও `report.txt` — হিসেবে ব্যাখ্যা হতে পারে, ফলে ভুল ফাইল মোছা বা error হতে পারে। সমাধান: সবসময় `"$file"` লেখা।

৫. Python-এ `subprocess.run()` -এ `shell=True` কেন এড়াতে হয়?
মডেল উত্তর: `shell=True` কমান্ড string-কে shell দিয়ে ব্যাখ্যা করে, ফলে ব্যবহারকারীর দেওয়া মানের মধ্যে `; rm -rf` বা `$(...)` থাকলে তা আলাদা কমান্ড হিসেবে চলতে পারে — command injection। নিরাপদ উপায়: argument list (`["ls", "-l", path]`) ও `shell=False` (ডিফল্ট), যাতে প্রতিটি argument আক্ষরিকভাবে program-এ যায়।

৬. API-তে pagination কেন থাকে এবং client-কে কী করতে হয়?
মডেল উত্তর: বড় তালিকা একবারে পাঠালে সার্ভার ও নেটওয়ার্কের উপর চাপ পড়ে, তাই API ফলাফলকে পৃষ্ঠায় ভাগ করে (যেমন `?page=2` বা cursor)। Client-কে next page-এর লিংক/সংকেত ধরে সব পৃষ্ঠা একত্র করতে হয়, প্রতিটি request-এ timeout ও status যাচাই করে এবং rate limit মেনে চলতে হয়।

৭. Cron job terminal-এ চলে কিন্তু cron-এ ব্যর্থ হয় — সবচেয়ে সাধারণ কারণগুলো কী?
মডেল উত্তর: cron ন্যূনতম environment নিয়ে চলে — PATH ছোট, বর্তমান directory আলাদা, ভেরিয়েবল নেই। সমাধান: script-এ সব কমান্ডের পূর্ণ পথ ব্যবহার করা, script-এর শুরুতে প্রয়োজনীয় ভেরিয়েবল সেট করা, stdout/stderr log ফাইলে redirect করা এবং `flock` দিয়ে ওভারল্যাপ ঠেকানো।

৮. SSH-এ host key বদলে গেলে সতর্কবার্তা এলে কী করা উচিত এবং কী করা উচিত নয়?
মডেল উত্তর: থামতে হবে ও কারণ খুঁজতে হবে — সার্ভার আবার install হয়েছে কি না, নাকি মাঝখানে কেউ ঢুকেছে। সার্ভার admin-এর সঙ্গে যাচাই করে তবেই `known_hosts` থেকে পুরনো key সরাতে হয়। যাচাই ছাড়া `StrictHostKeyChecking=no` দিয়ে এগিয়ে যাওয়া উচিত নয়, কারণ তাতে man-in-the-middle আক্রমণের ঝুঁকি থাকে।

৯. Idempotent automation বলতে কী বোঝায়? একটি উদাহরণ দিন।
মডেল উত্তর: একই কাজ একাধিকবার চালালে প্রতিবার একই ফল হয়, ক্ষতি হয় না। উদাহরণ: backup script-এ directory আগে থেকে থাকলে আবার তৈরি করার চেষ্টা না করা (`mkdir -p` বা `exist_ok=True`), অথবা user তৈরি করার আগে user আছে কি না পরীক্ষা করা। এতে cron-এ বারবার চলা নিরাপদ হয়।

১০. একটি automation tool-কে Bash-এর বদলে Python-এ লেখা কখন ভালো সিদ্ধান্ত?
মডেল উত্তর: যখন কাজে জটিল তথ্য কাঠামো (JSON/YAML), API call, জটিল শর্ত ও error handling, unit test বা একাধিক ফাইলের কাঠামো লাগে — তখন Python পড়তে সহজ, পরীক্ষা করা সহজ ও নিরাপদ। সহজ কমান্ড একত্র করা বা ছোট system কাজের জন্য Bash যথেষ্ট ও দ্রুত।

D.26 Git Command Interpretation — Git কমান্ড ব্যাখ্যা

১. `git log --oneline --graph --all --decorate`
উত্তর: সব branch-এর commit ইতিহাস সংক্ষেপে (এক লাইনে), branch/merge-এর গ্রাফ আকারে, branch ও tag-এর নামসহ দেখায়। `--graph` শাখা বিভাজন ও মিলন দেখায়, `--decorate` কোন commit-এ কোন branch/tag আছে তা লেখে। ব্যবহার: ইতিহাসের কাঠামো বোঝা।

২. `git diff --staged`

উত্তর: staging area-তে তোলা পরিবর্তন ও শেষ commit-এর মধ্যে পার্থক্য দেখায় — অর্থাৎ পরের commit-এ ঠিক কী যাবে। `git diff` (কোনো flag ছাড়া) দেখায় working directory ও staging area-র পার্থক্য। দুটি গুলিয়ে ফেললে ভুল জিনিস review হয়।

৩. `git checkout -b feature/login`

উত্তর: বর্তমান commit থেকে `feature/login` নামে নতুন branch তৈরি করে এবং তাতে চলে যায়। `git switch -c feature/login` একই কাজের আধুনিক রূপ। মনে রাখবেন: branch তৈরি হয় বর্তমান HEAD থেকে, main থেকে নয় — যদি main-এ না থাকেন, ভুল base থেকে branch তৈরি হবে।

৪. `git push --force-with-lease origin main`

উত্তর: local main-কে remote-এ জোর করে পাঠায়, তবে শর্ত থাকে: remote-এ শেষ দেখা অবস্থার পর কেউ নতুন commit পাঠায়নি। `--force`-এর চেয়ে নিরাপদ, কারণ অন্যের কাজ অজান্তে মুছে যাওয়ার ঝুঁকি কমে। তবু shared main-এ ব্যবহার করা উচিত নয় — শুধু নিজের branch-এ, ইতিহাস ঠিক করার পর।

৫. `git stash push -m "wip: config" --keep-index`

উত্তর: working directory-র unstaged পরিবর্তনগুলো সাময়িকভাবে সরিয়ে রাখে (stash), কিন্তু staged পরিবর্তন রেখে দেয় (`--keep-index`)। ফলে staged কাজ আলাদাভাবে commit বা পরীক্ষা করা যায়। পরে `git stash pop` দিয়ে ফিরিয়ে আনা যায়।

D.27 Bash Code Reading — Bash কোড পড়া

১. নিচের কোডের output কী হবে?

```
count=0
for f in *.txt; do
  count=$((count + 1))
done
echo "$count"
```

উত্তর: বর্তমান directory-তে যতগুলো `.txt` ফাইল আছে, সেই সংখ্যা। তবে কোনো `.txt` ফাইল না থাকলে (nullglob ছাড়া) pattern আক্ষরিক `*.txt` হিসেবে একবার চলবে, ফলে output হবে `1` — এটি একটি ফাঁদ; নিরাপদ উপায় `shopt -s nullglob` বা ফাইল আছে কি না পরীক্ষা করা।

২. নিচের কোডে কী ভুল আছে?

```
if [ $status = "ok" ]; then
  echo "fine"
fi
```

উত্তর: `$status` quote করা হয়নি। `status` খালি থাকলে `[ = "ok" ]` হয়ে syntax error হয়; space থাকলে ভেঙে যায়। সঠিক রূপ: `if [ "$status" = "ok" ]; then`। আধুনিক Bash-এ `[[ "$status" == "ok" ]]`-ও নিরাপদ।

৩. নিচের কোডটি কী করে?

```
while IFS= read -r line; do
  echo "$line"
done < report.txt
```

উত্তর: `report.txt`-এর প্রতিটি লাইন একে একে পড়ে ও ছাপে। `IFS=` লাইনের শুরু/শেষের space রক্ষা করে, `-r` backslash আক্ষরিক রাখে। `done < report.txt` ফাইলটিকে loop-এর input হিসেবে দেয় — pipe ব্যবহার করলে subshell সমস্যা (loop-এর ভেতরে সেট করা ভেরিয়েবল বাইরে হারিয়ে যাওয়া) হতে পারে।

৪. নিচের function-টি কী ফেরত দেয়?

```
check_disk() {
  df -h / | awk 'NR==2 {print $5}'
}
usage=$(check_disk)
echo "$usage"
```

উত্তর: root filesystem-এর ব্যবহৃত শতাংশ, যেমন `42%`। function stdout-এ print করে, আর `$(...)` সেই stdout ধরে `usage` ভেরিয়েবলে রাখে। মনে রাখবেন: function-এর `return` শুধু exit status দেয়; মান ফেরত দিতে এভাবেই stdout ব্যবহার করতে হয়।

৫. নিচের লাইনটি কেন বিপজ্জনক?

```
rm -rf $BACKUP_DIR/*
```

উত্তর: `$BACKUP_DIR` খালি বা unset থাকলে এটি হয়ে যায় `rm -rf /*` — অর্থাৎ পুরো filesystem মোছার চেষ্টা। নিরাপদ রূপ: `set -u` ব্যবহার করা, ভেরিয়েবল খালি কি না পরীক্ষা করা (`: "${BACKUP_DIR:?missing}"`), quote করা (`"$BACKUP_DIR"/*`) এবং মোছার আগে dry-run বা তালিকা দেখা।

D.28 Python Debugging Exercises — Python debugging অনুশীলন

১. ভুল কোড (ফাইল বন্ধ হয় না, exception-এ ফাইল খোলা থেকে যায়):

```
f = open("data.txt")
data = f.read()
print(data)
```

সংশোধিত কোড:

```
with open("data.txt", encoding="utf-8") as f:
    data = f.read()
print(data)
```

ব্যাখ্যা: প্রথম কোডে `f.close()` লেখা হয়নি; exception হলে ফাইল খোলাই থেকে যায়। `with` block শেষ হলে — এমনকি exception হলেও — ফাইল বন্ধ হয়। `encoding="utf-8"` দেওয়ায় বিভিন্ন পরিবেশে encoding নিয়ে সমস্যা হয় না।

২. ভুল কোড (KeyError):

```
config = {"host": "localhost"}
port = config["port"]
```

সংশোধিত কোড:

```
config = {"host": "localhost"}
port = config.get("port", 8080)
```

ব্যাখ্যা: `config["port"]` key না থাকলে `KeyError` তোলে। `.get()` key না থাকলে default মান (এখানে 8080) দেয়, ফলে প্রোগ্রাম থামে না। যদি অনুপস্থিত key আসলেই ত্রুটি হয়, তবে স্পষ্ট বার্তাসহ যাচাই করে থামাই ভালো।

৩. ভুল কোড (shell injection):

```
import subprocess
name = input("Enter filename: ")
subprocess.run("ls -l " + name, shell=True)
```

সংশোধিত কোড:

```
import subprocess
name = input("Enter filename: ")
subprocess.run(["ls", "-l", name], check=True)
```

ব্যাখ্যা: প্রথম কোডে ব্যবহারকারী `file.txt; rm -rf /`-এর মতো input দিলে shell তা আলাদা কমান্ড হিসেবে চালাবে। তালিকা আকারে argument দিলে (`shell=False`, ডিফল্ট) input শুধু একটি argument হিসেবে যায়। `check=True` ব্যর্থ হলে exception তোলে।

৪. ভুল কোড (timeout ছাড়া API call, exception ধরা হয় না):

```
import requests
resp = requests.get("https://api.example.com/health")
print(resp.json())
```

সংশোধিত কোড:

```
import requests
try:
    resp = requests.get("https://api.example.com/health", timeout=10)
    resp.raise_for_status()
    print(resp.json())
except requests.RequestException as exc:
    print(f"API call failed: {exc}")
```

ব্যাখ্যা: প্রথম কোডে সার্ভার সাড়া না দিলে প্রোগ্রাম অনির্দিষ্টকাল ঝুলে থাকে, আর 404/500 status-এও `.json()` ভুলভাবে চালানো হয়। সংশোধিত কোডে timeout আছে, `raise_for_status()` খারাপ status-এ exception তোলে এবং `RequestException` ধরে স্পষ্ট বার্তা দেওয়া হয়।

৫. ভুল কোড (mutable default argument):

```
def add_error(errors=[]):
    errors.append("disk full")
    return errors
```

সংশোধিত কোড:

```
def add_error(errors=None):
    if errors is None:
        errors = []
    errors.append("disk full")
    return errors
```

ব্যাখ্যা: Python-এ default list একবারই তৈরি হয় ও সব call-এ ভাগ হয় — ফলে প্রতিবার call করলে তালিকা বাড়তেই থাকে। `None` default দিয়ে ভেতরে নতুন list বানালে প্রতিটি call আলাদা তালিকা পায়। এটি Python-এর একটি বিখ্যাত ফাঁদ।

D.29 Script-Writing Challenges — স্ক্রিপ্ট লেখার চ্যালেঞ্জ

চ্যালেঞ্জ ১ (Beginner — Bash): Disk সতর্কতা স্ক্রিপ্ট। একটি Bash script লিখুন যা root filesystem-এর ব্যবহার শতাংশ পরীক্ষা করে: 80%-এর বেশি হলে WARNING, 90%-এর বেশি হলে CRITICAL বার্তা stderr-এ লিখে exit code 2 দিয়ে থামবে, নাহলে OK লিখে 0 দিয়ে থামবে।
সমাধানের রূপরেখা: `set -euo pipefail` দিয়ে শুরু করুন; `df --output=pcent / | tail -1 | tr -d ' %'` দিয়ে সংখ্যা নিন; সংখ্যা কি না যাচাই করুন; `-ge 90` ও `-ge 80` দিয়ে if/elif লিখুন; বার্তা stderr-এ (`>&2`) লিখুন; `shellcheck` দিয়ে যাচাই করুন।

চ্যালেঞ্জ ২ (Intermediate — Python): Log বিশ্লেষণ tool। একটি Python CLI লিখুন যা একটি log ফাইল পড়ে প্রতিটি log level (INFO, WARNING, ERROR) কতবার এসেছে, তা গোনে ও JSON-এ লিখে।
সমাধানের রূপরেখা: `argparse` দিয়ে input/output পথ নিন; `pathlib` দিয়ে ফাইল আছে কি না যাচাই করুন; `with open(..., encoding="utf-8")` দিয়ে লাইন ধরে পড়ুন; `collections.Counter` দিয়ে গুনুন; `json.dump()` দিয়ে ফল লিখুন; `logging` দিয়ে অগ্রগতি জানান; pytest দিয়ে একটি ছোট test লিখুন।

চ্যালেঞ্জ ৩ (Advanced — Bash + Python): Backup ও যাচাই। একটি tool বানান যা একটি directory-র `tar.gz` backup নেয়, SHA-256 checksum লেখে, checksum মিলিয়ে যাচাই করে এবং ফলাফল structured log-এ লেখে।
সমাধানের রূপরেখা: Bash wrapper-এ argument যাচাই, `mktemp -d` ও trap cleanup, `tar -czf`; Python অংশে `hashlib.sha256()` দিয়ে checksum গণনা, JSON log লেখা, exit code-ভিত্তিক সফল/ব্যর্থতা; dry-run flag যোগ করুন; idempotency রাখুন — একই backup আবার চালালে পুরনোটি নষ্ট হবে না।

D.30 Troubleshooting Case Studies — সমস্যা সমাধানের কেস স্টাডি

কেস ১: `git push` rejected — "non-fast-forward"।
লক্ষণ: push করলে Git বলে remote-এ আপনার local-এ নেই এমন commit আছে। নির্ণয়: অন্য কেউ (বা আপনি অন্য কম্পিউটার থেকে) main-এ push করেছে; আপনার local ইতিহাস পিছিয়ে আছে। সমাধান: প্রথমে `git fetch` করে দেখুন, তারপর `git pull --rebase` (নিজের branch-এ) বা `git pull` করে merge করুন; conflict থাকলে সমাধান করে commit করুন, তারপর push করুন। `--force` দিয়ে অন্যের কাজ মুছে ফেলবেন না।

কেস ২: Cron job চলে না, কিন্তু terminal-এ script চলে।
লক্ষণ: crontab-এ job আছে, কিন্তু log ফাইল তৈরি হয় না। নির্ণয়: (ক) cron-এর PATH-এ কমান্ড নেই — script-এ relative পথ বা `python` ব্যবহার করা হয়েছে; (খ) output redirect করা হয়নি, তাই ব্যর্থতা দেখা যাচ্ছে না; (গ) script-এ execute permission নেই বা shebang ভুল। সমাধান: সব পথ absolute করুন (`/usr/bin/python3`), crontab-এ `>> /var/log/job.log 2>&1` যোগ করুন, `chmod +x` ও shebang যাচাই করুন এবং `grep CRON /var/log/syslog` দিয়ে cron কী করেছে দেখুন।

কেস ৩: Python script-এ `ModuleNotFoundError: requests`, অথচ pip install করা হয়েছে।
লক্ষণ: terminal-এ `pip install requests` সফল, কিন্তু script চালালে module পাওয়া যায় না। নির্ণয়: pip ও python আলাদা পরিবেশের — system pip-এ install হয়েছে, কিন্তু script অন্য venv বা `python3`-এ চলছে। সমাধান: `which python3` ও `python3 -m pip --version` মিলিয়ে দেখুন; সঠিক venv activate করে `python -m pip install -r requirements.txt` চালান; script-এর shebang যাচাই করুন (`#!/usr/bin/env python3` ভুল venv ধরতে পারে)।

D.31 Practical Final Examination — ব্যবহারিক চূড়ান্ত পরীক্ষা

সময়: ৩ ঘণ্টা। মোট নম্বর: ১০০। পাস নম্বর: ৬০। পরিবেশ: Ubuntu Server 24.04 LTS, Git 2.43, Bash 5.2, Python 3.12। পরীক্ষার সময় ইন্টারনেট ব্যবহার করা যাবে, কিন্তু অন্যের কোড কপি করা যাবে না।

অংশ ক — MCQ পুনরালোচনা (২০ নম্বর)

অধ্যায় ১–২৪-এর ১২০টি MCQ থেকে ২০টি প্রশ্ন দেওয়া হবে (প্রতিটি ১ নম্বর)। প্রতিটি অধ্যায়-গোষ্ঠী (Git, Bash, Python, automation, workflow) থেকে প্রশ্ন থাকবে। উত্তরপত্রে শুধু সঠিক বিকল্প লিখলেই হবে, ব্যাখ্যা লাগবে না।

অংশ খ — হাতে-কলমে কাজ (৮০ নম্বর)

কাজ ১ — Git repository ও branch (২০ নম্বর): `exam-toolkit` নামে একটি repository তৈরি করুন; `README.md` ও `.gitignore` যোগ করে commit করুন; `feature/health-check` branch-এ একটি ফাইল যোগ করুন; main-এ একটি আলাদা commit করুন; branch-টি main-এ merge করুন; `v1.0.0` tag দিন; `git log --oneline --graph --all --decorate`-এর output দেখান।

কাজ ২ — Bash health script (২০ নম্বর): একটি Bash script লিখুন যা CPU load, memory ব্যবহার, disk ব্যবহার ও একটি service-এর status পরীক্ষা করে; ফলাফল পরিষ্কারভাবে ছাপে; কোনো পরীক্ষা ব্যর্থ হলে nonzero exit code দেয়; `set -euo pipefail`, quoting ও input validation ব্যবহার করে; `shellcheck`-এ কোনো error থাকে না।

কাজ ৩ — Python API ও logging tool (২০ নম্বর): একটি Python CLI লিখুন যা একটি config ফাইল (JSON/YAML) পড়ে, একটি public HTTP API-তে timeout ও error handling-সহ request পাঠায়, JSON উত্তর প্রক্রিয়া করে, ফলাফল ফাইলে লেখে এবং structured logging ব্যবহার করে। venv ও `requirements.txt` থাকতে হবে; secret code-এ লেখা যাবে না।

কাজ ৪ — Cron job সেটআপ (২০ নম্বর): কাজ ২-এর script-টি প্রতি ৩০ মিনিটে চালানোর জন্য crontab entry লিখুন (absolute পথসহ), output log ফাইলে redirect করুন, `flock` দিয়ে ওভারল্যাপ ঠেকান এবং cron expression-এর প্রতিটি field-এর অর্থ ব্যাখ্যা করুন।

মূল্যায়নের রুব্রিক

মানদণ্ড	চমৎকার (৯০–১০০%)	ভালো (৭০–৮৯%)	পাস (৬০–৬৯%)	অসম্পূর্ণ (৬০%-এর নিচে)
সঠিকতা	সব কাজ সঠিকভাবে চলে, output যাচাই করা যায়	ছোট ভুল আছে, মূল কাজ চলে	মূল কাজ আংশিক চলে	কাজ চলে না বা ভুল ফল দেয়
নিরাপত্তা	quoting, shell=False, secret বাইরে, permission সঠিক	ছোট একটি নিরাপত্তা ঘাটতি	একাধিক ঘাটতি, কিন্তু বিপজ্জনক নয়	injection বা secret ফাঁসের ঝুঁকি
			আংশিক handling	

Error handling	ব্যর্থতা log হয়, exit code সঠিক, cleanup হয়	বেশিরভাগ ব্যর্থতা ধরা হয়		ব্যর্থতা চুপচাপ পেরিয়ে যায়
গঠন ও নথি	modular কোড, README, পরিষ্কার নাম	মোটামুটি গঠন, কিছু নথি	এক ফাইলে কাজ, নথি কম	অগোছালো, নথি নেই
পরীক্ষা	pytest/ShellCheck চালানো হয়েছে, প্রমাণ আছে	কিছু পরীক্ষা আছে	হাতে যাচাইয়ের উল্লেখ আছে	কোনো যাচাই নেই

> **WARNING:** পরীক্ষায় কখনো অন্যের repository-তে `--force` push করবেন না, production সার্ভারে পরীক্ষামূলক script চালাবেন না এবং কোনো secret (token, password) উত্তরপত্রে লিখবেন না।

পরিশিষ্ট E • Glossary

Appendix E: Glossary — পরিভাষা (English–Bangla)

এই পরিশিষ্টে বইয়ে ব্যবহৃত ১৯৭টি গুরুত্বপূর্ণ পরিভাষা ইংরেজি বর্ণানুক্রমে দেওয়া হয়েছে। প্রতিটি শব্দের পাশে এক বা দুই লাইনে বাংলায় অর্থ লেখা হয়েছে, যাতে পড়ার সময় দ্রুত দেখে নেওয়া যায়।

Term	বাংলা অর্থ
API (Application Programming Interface)	দুটি প্রোগ্রামের মধ্যে তথ্য আদান-প্রদানের নিয়ম ও মাধ্যম; যেমন একটি সার্ভার থেকে তথ্য নেওয়ার নির্দিষ্ট পথ।
Argument	কমান্ড বা function-কে দেওয়া মান; যেমন `ls -l`-এ `-l` একটি argument।
Array	একটি নামের অধীনে সাজানো মানের তালিকা; Bash-এ `files=(a.txt b.txt)` একটি array।
Artifact	কোনো কাজের ফল হিসেবে তৈরি ফাইল বা প্যাকেজ; যেমন build-এর পর তৈরি হওয়া ফাইল।
Authentication	পরিচয় যাচাই করা — আপনি আসলেই সেই ব্যবহারকারী কি না, তা প্রমাণ করা।
Authorization	অনুমতি যাচাই করা — পরিচয় প্রমাণিত ব্যবহারকারী কী করতে পারবে, তা নির্ধারণ করা।
Automation	মানুষের বারবার হস্তক্ষেপ ছাড়া কাজ স্বয়ংক্রিয়ভাবে করানো।
awk	কলামভিত্তিক text প্রক্রিয়াকরণের শক্তিশালী Linux tool; field আলাদা করে হিসাব করা যায়।
Backup	তথ্য হারালে ফিরিয়ে আনার জন্য রাখা অনুলিপি।
Bash	Linux-এর জনপ্রিয় shell ও scripting ভাষা; এই বইয়ের প্রধান scripting ভাষা।
Base branch	যে branch থেকে নতুন branch তৈরি হয় বা যেখানে merge করা হয়; সাধারণত main।
Binary	শুধু ০ ও ১ দিয়ে প্রকাশিত তথ্য; মানুষের পড়ার মতো text নয় এমন ফাইল।
Blob	Git object, যা একটি ফাইলের বিষয়বস্তু সংরক্ষণ করে; নাম থাকে না, শুধু content থাকে।
Branch	একটি commit-কে নির্দেশ করা pointer; মূল কাজ থেকে আলাদা করে কাজ করার পথ।
Break	loop থেকে সঙ্গে সঙ্গে বেরিয়ে আসার নির্দেশ।
Bug	প্রোগ্রামের ভুল, যার ফলে ভুল ফলাফল বা অপ্রত্যাশিত আচরণ হয়।
Build	source code থেকে চালানোর উপযোগী প্রোগ্রাম বা প্যাকেজ তৈরির প্রক্রিয়া।
Cache	বারবার লাগে এমন তথ্য দ্রুত পাওয়ার জন্য রাখা সাময়িক সংরক্ষণ।
Callback	কোনো ঘটনা ঘটলে পরে চালানোর জন্য দেওয়া function।
Case statement	Bash-এ একই মানের অনেকগুলো pattern-এর মধ্যে একটিকে বেছে নেওয়ার গঠন।

Certificate	পরিচয় ও নিরাপত্তা প্রমাণের ডিজিটাল নথি; যেমন TLS certificate।
Cherry-pick	অন্য branch থেকে নির্দিষ্ট একটি commit বেছে এনে বর্তমান branch-এ প্রয়োগ করা।
CI/CD	Continuous Integration / Continuous Delivery — কোড যাচাই ও প্রকাশ স্বয়ংক্রিয় করার পদ্ধতি।
CLI (Command-Line Interface)	লেখা কমান্ড দিয়ে প্রোগ্রাম চালানোর মাধ্যম; GUI-এর বিপরীত।
Clone	remote repository-র সম্পূর্ণ অনুলিপি নিজের কম্পিউটারে নেওয়া।
Cloud	ইন্টারনেটের মাধ্যমে ভাড়া নেওয়া computing সেবা (সার্ভার, storage ইত্যাদি)।
Code review	merge-এর আগে অন্য developer-এর কোড পড়ে ভুল, শৈলী ও নিরাপত্তা যাচাই করা।
Commit	Git-এ staging area-র snapshot স্থায়ীভাবে ইতিহাসে সংরক্ষণ করা।
Commit message	commit-এর সঙ্গে লেখা সংক্ষিপ্ত বিবরণ — কী বদলেছে ও কেন।
Compiler	পুরো source code একবারে machine code-এ রূপান্তরকারী প্রোগ্রাম।
Condition	সত্য/মিথ্যা যাচাইয়ের শর্ত, যার উপর ভিত্তি করে if/while সিদ্ধান্ত নেয়।
Configuration	পরিবেশভেদে বদলায় এমন সেটিং (URL, পথ, সীমা) — code থেকে আলাদা রাখা হয়।
Conflict	একই জায়গায় দুটি ভিন্ন পরিবর্তন থাকায় Git-এর স্বয়ংক্রিয় merge না পারা।
Container	অ্যাপ্লিকেশন ও তার dependency-কে একসঙ্গে প্যাকেজ করার হালকা বিচ্ছিন্ন পরিবেশ।
Continue	loop-এর বর্তমান চক্র বাদ দিয়ে পরের চক্রে যাওয়ার নির্দেশ।
Cron	Linux-এ নির্দিষ্ট সময়ে কাজ চালানোর সময়সূচি ব্যবস্থা।
Crontab	cron job-এর তালিকা ধারণকারী ফাইল; প্রতিটি লাইনে সময় ও কমান্ড থাকে।
Curl	command line থেকে HTTP request পাঠানোর tool।
Daemon	পেছনে চলমান সেবা প্রক্রিয়া; যেমন web server বা cron daemon।
Dashboard	system-এর অবস্থা এক নজরে দেখার সারসংক্ষেপ পর্দা।
Database	কাঠামোবদ্ধভাবে তথ্য সংরক্ষণ ও খোঁজার ব্যবস্থা।
Debugging	প্রোগ্রামের ভুল খুঁজে বের করে ঠিক করার প্রক্রিয়া।
Default value	মান না দিলে স্বয়ংক্রিয়ভাবে ব্যবহৃত মান।
Dependency	আপনার প্রোগ্রাম চালাতে প্রয়োজনীয় অন্য library বা tool।
Deployment	কোড সার্ভারে বসিয়ে ব্যবহারকারীদের জন্য চালু করা।
Detached HEAD	HEAD যখন কোনো branch-এর দিকে নয়, সরাসরি কোনো commit-কে নির্দেশ করে — এমন অবস্থা।

Dictionary	Python-এ key–value জোড়ায় তথ্য রাখার কাঠামো; যেমন `{"host": "localhost"}`।
Directory	ফাইল সাজিয়ে রাখার ফোল্ডার।
Disk usage	disk-এর কতটুকু জায়গা ব্যবহৃত হয়েছে, তার পরিমাপ।
Distributed VCS	প্রতিটি অনুলিপিতে পূর্ণ ইতিহাস থাকে এমন version control ব্যবস্থা; যেমন Git।
Docker	container তৈরি ও চালানোর জনপ্রিয় platform।
Documentation	কোনো tool বা প্রজেক্ট কীভাবে ব্যবহার করতে হয়, তার লিখিত নির্দেশনা।
Dry run	আসল পরিবর্তন না করে কী করা হতো, তা দেখানো — নিরাপদ যাচাইয়ের পদ্ধতি।
Encryption	তথ্যকে এমনভাবে বদলে দেওয়া, যাতে শুধু অনুমোদিত পক্ষ পড়তে পারে।
Endpoint	API-র নির্দিষ্ট ঠিকানা, যেখানে request পাঠাতে হয়; যেমন `/api/users`।
Environment variable	operating system বা shell থেকে প্রোগ্রামে পৌঁছে দেওয়া সেটিং মান; যেমন `HOME`।
Error handling	ত্রুটি ঘটলে প্রোগ্রাম ভেঙে না পড়ে নিরাপদে সামলানোর ব্যবস্থা।
Exception	Python-এ চলার সময় ওঠা ত্রুটি, যা try/except দিয়ে ধরা যায়।
Exit code	প্রোগ্রাম শেষ হওয়ার সময় ফেরত দেওয়া সংখ্যা; 0 মানে সফল, nonzero মানে ব্যর্থতা।
Export	Bash-এ ভেরিয়েবলকে child process-এর কাছে পৌঁছে দেওয়ার নির্দেশ।
Fast-forward merge	main-এ নতুন commit না থাকলে pointer শুধু এগিয়ে দিয়ে merge করা; merge commit তৈরি হয় না।
Fetch	remote-এর পরিবর্তন ডাউনলোড করা, কিন্তু working directory না বদলানো।
Field separator	text-কে field-এ ভাগ করার চিহ্ন; যেমন space, comma বা colon।
File descriptor	খোলা ফাইল বা stream-এর সংখ্যাগত পরিচয়; 0 = stdin, 1 = stdout, 2 = stderr।
File permission	কে ফাইল পড়তে, লিখতে বা চালাতে পারবে, তার নিয়ম; যেমন 644, 755।
Fixture	pytest-এ test-এর জন্য আগে থেকে তৈরি করা তথ্য বা পরিবেশ।
Fork	GitHub-এ অন্যের repository-র নিজস্ব অনুলিপি তৈরি করা।
Function	নাম দিয়ে ডাকা যায় এমন পুনর্ব্যবহারযোগ্য কোডের অংশ।
Git	distributed version control ব্যবস্থা; পরিবর্তনের ইতিহাস সংরক্ষণ করে।
GitHub	Git repository হোস্ট করা ও collaboration-এর অনলাইন সেবা।
Gitignore	Git যে ফাইলগুলো track করবে না, তার তালিকা ধারণকারী `.gitignore` ফাইল।
Glob	ফাইলের নাম মেলানোর pattern; যেমন `*.log`।

grep	ফাইলে নির্দিষ্ট pattern খোঁজার Linux tool।
Hash	তথ্য থেকে তৈরি নির্দিষ্ট দৈর্ঘ্যের অনন্য মান; Git-এ প্রতিটি object-এর পরিচয়।
HEAD	বর্তমান branch বা commit-কে নির্দেশ করা Git pointer।
Health check	সার্ভার বা সেবা ঠিকমতো চলছে কি না, তা পরীক্ষা করা।
Host	নেটওয়ার্কে যুক্ত কম্পিউটার বা সার্ভার।
HTTP	ওয়েবে তথ্য আদান-প্রদানের protocol।
HTTPS	TLS দিয়ে encrypt করা HTTP; নিরাপদ ওয়েব যোগাযোগের মান।
Idempotency	একই কাজ একাধিকবার চালালেও একই ফল পাওয়ার গুণ।
Index (staging area)	পরের commit-এ যাবে এমন পরিবর্তন রাখার Git-এর অস্থায়ী জায়গা।
Infrastructure	সার্ভার, নেটওয়ার্ক ও storage-সহ কোনো ব্যবস্থা চালানোর ভিত্তিগত কাঠামো।
Input validation	ব্যবহারকারীর দেওয়া তথ্য ব্যবহারের আগে তা সঠিক ও নিরাপদ কি না যাচাই করা।
Integration test	একাধিক অংশ একসঙ্গে কাজ করে কি না, তা পরীক্ষা করা।
Interpreter	source code লাইন ধরে ধরে অনুবাদ করে চালানো প্রোগ্রাম; যেমন Python interpreter।
Issue	GitHub-এ bug, কাজ বা আলোচনা track করার item।
JSON	JavaScript Object Notation — হালকা, মানুষের পড়ার মতো তথ্য বিনিময়ের ফরম্যাট।
Key pair	public key ও private key-এর জোড়া; SSH authentication-এ ব্যবহৃত হয়।
Known hosts	SSH-এ আগে যাচাই করা সার্ভারের host key-এর তালিকা (`~/.ssh/known_hosts`)।
Library	পুনর্ব্যবহারযোগ্য function-এর সংগ্রহ; যেমন Python-এর requests library।
Linting	কোড চালানোর আগেই ভুল ও শৈলীর সমস্যা খুঁজে বের করা।
Linux	open-source operating system kernel ও তার উপর ভিত্তি করে তৈরি OS পরিবার।
Local variable	শুধু একটি function-এর ভেতরে কার্যকর ভেরিয়েবল।
Lock file	একই কাজ একসঙ্গে দুবার চলা ঠেকাতে ব্যবহৃত ফাইল।
Log level	log বার্তার গুরুত্বের স্তর; যেমন DEBUG, INFO, WARNING, ERROR, CRITICAL।
Logging	প্রোগ্রাম কী করছে, তার সময়সহ লিখিত রেকর্ড রাখা।
Loop	একই কাজ বারবার করার গঠন; যেমন for ও while।
Main branch	repository-র প্রধান branch; সাধারণত স্থিতিশীল ও deployable অবস্থায় রাখা হয়।
Man page	Linux কমান্ডের আনুষ্ঠানিক নির্দেশিকা; `man ls` দিয়ে দেখা যায়।

Merge	এক branch-এর পরিবর্তন অন্য branch-এ একত্র করা।
Merge conflict	একই লাইনে ভিন্ন পরিবর্তন থাকায় merge স্বয়ংক্রিয়ভাবে না হওয়া।
Mock	test-এ আসল বস্তুর বদলে ব্যবহৃত নকল বস্তু; বাইরের সেবা ছাড়া test চালানো যায়।
Module	Python-এ পুনর্ব্যবহারযোগ্য কোড ধারণকারী ফাইল।
Monitoring	system-এর অবস্থা নিয়মিত পর্যবেক্ষণ করা।
Mutable	তৈরির পর বদলানো যায় এমন; যেমন Python list।
Namespace	নামগুলোকে আলাদা গোষ্ঠীতে ভাগ করার ব্যবস্থা, যাতে নামের সংঘাত না হয়।
Networking	কম্পিউটারগুলোর মধ্যে সংযোগ ও তথ্য আদান-প্রদানের ব্যবস্থা।
Nullglob	Bash option, যাতে কোনো ফাইল না মিললে glob pattern খালি হয়ে যায়, আক্ষরিক থাকে না।

পরিশিষ্ট E (চলমান) • Glossary — O–Z

Glossary (চলমান): O–Z

Term	বাংলা অর্থ
Object database	Git-এর `.git/objects` — blob, tree ও commit object সংরক্ষণের জায়গা।
Observability	log, metric ও status দিয়ে system-এর ভেতরের অবস্থা বোঝার ক্ষমতা।
Open source	যে software-এর source code সবার জন্য উন্মুক্ত ও ব্যবহারযোগ্য।
Operating system	hardware ও প্রোগ্রামের মধ্যে সমন্বয়কারী software; যেমন Ubuntu।
Package	install করার উপযোগী software-এর বান্ডিল; Python-এ pip দিয়ে install করা হয়।
Pagination	বড় ফলাফলকে ছোট ছোট পৃষ্ঠায় ভাগ করে দেওয়া; API-তে সাধারণ।
Parameter	function বা কমান্ডে পাঠানো মানের নাম; argument হলো তার বাস্তব মান।
Parent commit	কোনো commit-এর ঠিক আগের commit; ইতিহাসের শৃঙ্খল তৈরি করে।
Password	পরিচয় প্রমাণের গোপন শব্দ; কখনো code বা Git-এ লেখা যাবে না।
Path	ফাইল বা directory-র অবস্থান; যেমন `/var/log/syslog`।
PATH variable	shell কমান্ড খুঁজতে যে directory-গুলো দেখে, তার তালিকা।
Permission	কে কী করতে পারবে, তার নিয়ম; ফাইল, directory ও সেবায় প্রযোজ্য।
Pipeline	এক কমান্ডের output অন্য কমান্ডের input হিসেবে দেওয়ার শৃঙ্খল; যেমন `ls | grep txt`।
Pipefail	Bash option, যাতে pipeline-এর যেকোনো অংশ ব্যর্থ হলে পুরো pipeline ব্যর্থ ধরা হয়।
Port	নেটওয়ার্কে নির্দিষ্ট সেবা চেনার সংখ্যা; যেমন SSH-এর 22, HTTPS-এর 443।
Process	চলমান প্রোগ্রামের একটি instance; প্রতিটির নিজস্ব process ID থাকে।
Production	আসল ব্যবহারকারীরা ব্যবহার করে এমন live পরিবেশ।
Pull	remote-এর পরিবর্তন এনে local branch-এ মেলানো (fetch + merge)।
Pull request	এক branch-এর পরিবর্তন অন্য branch-এ merge করার প্রস্তাব ও review-এর জায়গা।
Push	local commit remote repository-তে পাঠানো।
pytest	Python-এর জনপ্রিয় testing framework; `test_` দিয়ে শুরু হওয়া function চালায়।
Python	সহজে পড়া যায় এমন জনপ্রিয় প্রোগ্রামিং ভাষা; এই বইয়ের automation ভাষা।
Rate limiting	নির্দিষ্ট সময়ে কতগুলো request করা যাবে, তার সীমা; বেশি হলে 429 status আসে।
Read-only	শুধু পড়া যায়, বদলানো যায় না এমন অবস্থা।

Rebase	commit-গুলোকে নতুন base-এর উপর আবার প্রয়োগ করে সরলরৈখিক ইতিহাস বানানো।
Recovery	ব্যর্থতা বা তথ্য হারানোর পর আগের সঠিক অবস্থায় ফেরা।
Redirect	কমান্ডের output বা error-কে পর্দার বদলে ফাইলে বা অন্য জায়গায় পাঠানো।
Reference (ref)	Git-এ commit-কে নির্দেশ করা নাম; যেমন branch ও tag।
Refactoring	আচরণ না বদলে কোডের গঠন পরিষ্কার ও উন্নত করা।
Regression test	নতুন পরিবর্তনে আগের কাজ ভেঙে গেছে কি না, তা যাচাই করার test।
Regular expression	text-এর pattern মেলানোর সংক্ষিপ্ত ভাষা; যেমন `^[0-9]+$`।
Release	ব্যবহারের জন্য প্রকাশিত software-এর নির্দিষ্ট সংস্করণ।
Remote	অন্য জায়গায় থাকা repository-র নাম; সাধারণত `origin`।
Repository	Git-এ প্রজেক্টের ফাইল ও পূর্ণ ইতিহাস ধারণকারী ভাণ্ডার।
Request	client থেকে সার্ভারের কাছে তথ্য চাওয়ার বার্তা।
Response	request-এর উত্তরে সার্ভারের পাঠানো বার্তা ও তথ্য।
Retry	ব্যর্থ কাজ নির্দিষ্ট নিয়মে আবার চেষ্টা করা।
Return code	exit code-এর আরেক নাম; function বা প্রোগ্রামের সমাপ্তির সংকেত।
Rollback	ব্যর্থ deployment বা পরিবর্তনের পর আগের সঠিক সংস্করণে ফেরা।
Root user	Linux-এর সর্বোচ্চ ক্ষমতাসম্পন্ন ব্যবহারকারী; অসতর্ক ব্যবহার বিপজ্জনক।
rsync	শুধু বদলে যাওয়া অংশ কপি করে ফাইল/ফোল্ডার sync করার tool।
Runtime	প্রোগ্রাম চলার সময়ের পরিবেশ ও অবস্থা।
scp	SSH দিয়ে নিরাপদে ফাইল কপি করার কমান্ড।
Script	কমান্ড বা নির্দেশনার লিখিত ফাইল, যা চালালে কাজগুলো একে একে হয়।
SDK	Software Development Kit — নির্দিষ্ট platform-এ কাজ করার tool ও library-র সংগ্রহ।
Secret	API key, token বা password-এর মতো গোপন তথ্য; code-এ লেখা যাবে না।
sed	stream editor — text-এর নির্দিষ্ট অংশ বদলানোর Linux tool।
Semantic versioning	major.minor.patch নিয়মে সংস্করণ দেওয়া; যেমন 2.3.1।
Server	অন্য কম্পিউটারকে সেবা দেওয়া কম্পিউটার বা প্রোগ্রাম।
Set -e	Bash option — কোনো কমান্ড ব্যর্থ হলে script থেমে যাবে।
Set -u	Bash option — অনির্ধারিত ভেরিয়েবল ব্যবহার করলে error হবে।
Shell	ব্যবহারকারীর কমান্ড পড়ে operating system-কে চালাতে বলা প্রোগ্রাম; যেমন Bash।

ShellCheck	Bash script-এর ভুল ধরার static analysis tool।
Shell injection	আক্রমণকারী input-এর মাধ্যমে shell-এ নিজের কমান্ড চালানোর আক্রমণ।
Signal	process-কে পাঠানো বার্তা; যেমন SIGTERM (থামতে বলা) বা SIGINT (Ctrl+C)।
Snapshot	কোনো নির্দিষ্ট সময়ে পুরো প্রজেক্টের অবস্থার অনুলিপি; Git commit-এর ভিত্তি।
Sorting	তথ্যকে নির্দিষ্ট ক্রমে সাজানো; যেমন `sort` কমান্ড।
Source code	মানুষের লেখা প্রোগ্রামের মূল লেখা।
SSH	Secure Shell — দূরের কম্পিউটারে encrypt করা নিরাপদ সংযোগের protocol।
SSH agent	private key মনে রেখে বারবার passphrase না চেয়ে SSH-এ সাহায্য করা প্রোগ্রাম।
Staging area	পরের commit-এ যাবে এমন পরিবর্তন রাখার Git-এর জায়গা; index নামেও পরিচিত।
Standard error (stderr)	ত্রুটি ও সতর্কবার্তা পাঠানোর stream; file descriptor 2।
Standard input (stdin)	প্রোগ্রামে তথ্য ঢোকানোর stream; file descriptor 0।
Standard output (stdout)	প্রোগ্রামের আসল ফলাফল পাঠানোর stream; file descriptor 1।
Status code	HTTP উত্তরের সংখ্যাগত সংকেত; যেমন 200 সফল, 404 পাওয়া যায়নি, 500 সার্ভার ত্রুটি।
String	অক্ষরের ক্রম; যেমন `"hello"`।
Subprocess	Python থেকে চালানো আলাদা process; `subprocess` module দিয়ে নিয়ন্ত্রণ করা হয়।
sudo	সাধারণ ব্যবহারকারীকে সাময়িকভাবে root ক্ষমতায় কমান্ড চালানোর অনুমতি দেওয়া।
systemd	আধুনিক Linux-এ সেবা ও system পরিচালনার ব্যবস্থা।
systemd timer	cron-এর বিকল্প হিসেবে systemd দিয়ে সময়মতো কাজ চালানোর ব্যবস্থা।
Tag	নির্দিষ্ট commit-কে চিহ্নিত করা স্থির নাম; যেমন `v1.0.0`।
Test	কোড ঠিকমতো কাজ করে কি না, তা যাচাই করার প্রক্রিয়া বা কোড।
Timeout	কোনো কাজের জন্য অপেক্ষার সর্বোচ্চ সময়; এরপর ব্যর্থ ধরা হয়।
Token	API-তে পরিচয় প্রমাণের গোপন string; password-এর মতোই সুরক্ষিত রাখতে হয়।
Traceback	Python-এ exception-এর সময় দেখানো ভুলের উৎসের ধারাবাহিক তালিকা।
Tree object	Git object, যা directory-র তালিকা ধারণ করে — কোন নামে কোন blob বা sub-tree আছে।
Troubleshooting	সমস্যার কারণ খুঁজে ধাপে ধাপে সমাধান করার প্রক্রিয়া।
Tuple	Python-এ অপরিবর্তনীয় (immutable) মানের ক্রম; যেমন `(1, 2)`।
Unit test	কোডের একটি ছোট অংশ একা পরীক্ষা করা।
Untracked file	Git এখনো track করে না এমন ফাইল; `git status`-এ দেখা যায়।

User input	ব্যবহারকারীর দেওয়া তথ্য; যাচাই ছাড়া ব্যবহার করা নিরাপদ নয়।
Variable	মান ধারণ করার নামযুক্ত জায়গা; যেমন `name="Nahid"`।
Virtual environment	Python প্রজেক্টের dependency আলাদা রাখার বিচ্ছিন্ন পরিবেশ (venv)।
Version control	ফাইলের পরিবর্তনের ইতিহাস সংরক্ষণ ও পরিচালনার ব্যবস্থা।
Webhook	কোনো ঘটনা ঘটলে সার্ভার থেকে অন্য সার্ভারে স্বয়ংক্রিয়ভাবে পাঠানো HTTP বার্তা।
While loop	শর্ত সত্য থাকা পর্যন্ত বারবার চলা loop।
Working directory	বর্তমানে কাজ করা directory; Git-এ ফাইল সম্পাদনার জায়গা।
YAML	মানুষের পড়ার উপযোগী structured data ফরম্যাট; config ফাইলে ব্যবহৃত হয়।
Zip archive	একাধিক ফাইল compress করে এক ফাইলে রাখার পদ্ধতি; যেমন `.zip` বা `.tar.gz`।

মোট পরিভাষা: এই পরিশিষ্টে ১৯৭টি term আছে — A–N অংশে ১০৮টি এবং O–Z অংশে ৮৯টি।

পরিশিষ্ট F • References

Appendix F: References — তথ্যসূত্র

এই বইয়ের সব কমান্ড, syntax ও আচরণ নিচের আনুষ্ঠানিক (official) নথিগুলোর সঙ্গে মিলিয়ে যাচাই করা হয়েছে। প্রকাশের সময় ব্যবহৃত পরিবেশ: Ubuntu Server 24.04 LTS, Git 2.43, Bash 5.2, Python 3.12। কোনো তথ্য নিয়ে সন্দেহ হলে প্রথমে এই উৎসগুলো দেখুন — কোনো ব্লগ বা ভিডিও নয়।

#	উৎস	URL	কী পাবেন
1	Git Documentation	https://git-scm.com/doc	Git-এর আনুষ্ঠানিক নথি ও সব কমান্ডের ব্যাখ্যা।
2	Pro Git Book	https://git-scm.com/book/en/v2	Git-এর ভেতরের কাজ, branching ও workflow-এর পূর্ণাঙ্গ বই।
3	GitHub Docs	https://docs.github.com	GitHub repository, pull request, issue ও নিরাপত্তার নথি।
4	GitHub — About Pull Requests	https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests	Pull request কী ও কীভাবে review হয়, তার ব্যাখ্যা।
5	GitHub — SSH Keys	https://docs.github.com/en/authentication/connecting-to-github-with-ssh	SSH key তৈরি ও GitHub-এ যোগ করার ধাপ।
6	GNU Bash Manual	https://www.gnu.org/software/bash/manual/	Bash-এর আনুষ্ঠানিক manual — variable, loop, function ও option।
7	GNU Coreutils Manual	https://www.gnu.org/software/coreutils/manual/	ls, cp, grep, sort, cut, find-সহ মূল Linux utility-র নথি।
8	Bash Reference — Shell Expansions	https://www.gnu.org/software/bash/manual/html_node/Shell-Expansions.html	quoting, word splitting ও command substitution-এর নিয়ম।
9	Python Documentation (3.12)	https://docs.python.org/3/	Python ভাষা, library ও tutorial-এর আনুষ্ঠানিক নথি।
10	Python — subprocess	https://docs.python.org/3/library/subprocess.html	subprocess দিয়ে নিরাপদে কমান্ড চালানো ও output ধরা।
11	Python — venv	https://docs.python.org/3/library/venv.html	virtual environment তৈরি ও ব্যবহারের নথি।

12	Python — logging	https://docs.python.org/3/library/logging.html	logging module, level ও handler-এর ব্যাখ্যা।
13	Python — argparse	https://docs.python.org/3/library/argparse.html	command-line argument parsing-এর নথি।
14	Python — pathlib	https://docs.python.org/3/library/pathlib.html	pathlib দিয়ে ফাইল ও directory নিয়ে কাজের নথি।
15	Python — json	https://docs.python.org/3/library/json.html	JSON পড়া ও লেখার নথি।
16	Python — shutil	https://docs.python.org/3/library/shutil.html	ফাইল কপি, সরানো ও archive করার নথি।
17	requests Documentation	https://requests.readthedocs.io/	Python-এ HTTP request পাঠানো, timeout ও error handling।
18	PyYAML Documentation	https://pyyaml.org/wiki/PyYAMLDocumentation	YAML পড়া ও লেখার নথি।
19	pytest Documentation	https://docs.pytest.org/	pytest দিয়ে test লেখা, fixture ও mocking-এর নথি।
20	ShellCheck	https://www.shellcheck.net/	Bash script যাচাইয়ের tool ও তার সতর্কবার্তার ব্যাখ্যা।
21	ShellCheck (GitHub)	https://github.com/koalaman/shellcheck	ShellCheck-এর source code ও প্রতিটি সতর্কবার্তার তালিকা।
22	Ubuntu Server Documentation	https://ubuntu.com/server/docs	Ubuntu Server install, পরিচালনা ও সেবার নথি।
23	OpenSSH Manual	https://www.openssh.com/manual.html	SSH, sshd ও ssh-keygen-এর আনুষ্ঠানিক নথি।
24	man7.org — Linux man-pages	https://man7.org/linux/man-pages/	Linux man page-গুলোর অনলাইন সংগ্রহ; যেমন cron, bash, ssh।
25	man7.org — crontab(5)	https://man7.org/linux/man-pages/man5/crontab.5.html	crontab ফাইলের field ও syntax-এর আনুষ্ঠানিক ব্যাখ্যা।
26	systemd — Timers	https://www.freedesktop.org/software/systemd/man/latest/systemd.timer.html	systemd timer দিয়ে সময়মতো কাজ চালানোর নথি।
27	systemd — systemctl	https://www.freedesktop.org/software/systemd/man/latest/systemctl.html	সেবা দেখা, চালু ও বন্ধ করার systemctl-এর নথি।

28	Git — gitignore Documentation	https://git-scm.com/docs/gitignore	.gitignore pattern-এর নিয়ম।
29	Git — git-branch Documentation	https://git-scm.com/docs/git-branch	branch তৈরি, তালিকা ও মোছার নথি।
30	Git — git-rebase Documentation	https://git-scm.com/docs/git-rebase	rebase-এর ব্যবহার ও সতর্কতার নথি।

> **CONCEPT:** এই বইয়ের কোনো উদাহরণ চালাতে গিয়ে ভিন্ন ফল পেলে প্রথমে সংস্করণ মিলিয়ে দেখুন (Git 2.43, Bash 5.2, Python 3.12) — তারপর উপরের আনুষ্ঠানিক নথিতে সংশ্লিষ্ট পৃষ্ঠাটি খুলুন। সংস্করণ বদলালে আচরণ বদলাতে পারে, কিন্তু নীতিগুলো একই থাকে।

DEVOPS ENGINEERING MASTERY — Volume 03: Git, Scripting & Automation Engineering — First Edition, 8 November 2026। পরীক্ষিত পরিবেশ: Ubuntu Server 24.04 LTS, Git 2.43, Bash 5.2, Python 3.12।